MAKERERE  **UNIVERSITY**

COLLEGE OF COMPUTING AND INFORMATION SCIENCES
EMBEDDED SYSTEMS

An ESP32-Based RFID Access Control, Occupancy Monitoring and Automated Lecture Room Lighting System for CoCIS

Report

Embedded Systems, Department of Networks, Makerere University

Name	Registration number
Kavuma Mustafa Kasirye	24/U/05533/PS
Otimong Jonathan	24/U/10833/PS
Nakabuubi Hilda Joyce	24/U/08182/EVE
Okoth Victor	23/U/1315
Nyonyozi Maria Lisa Loyce	23/U/16424/EVE

Lecturer / Supervisor: DR. MARY NSABAGWA

Group number: Group 13

WOKWI SIMULATION LINK:https://wokwi.com/projects/479582612769517569

FIRMWARE LINK:https://raw.githubusercontent.com/otimongjonathan/firmware/main/merged-firmware.bin

TASKS COMPLETED BY EACH MEMBER

Member	Role	Link to video
Kavuma Mustafa Kasirye	Lighting system	room lighting presentation
Otimong Jonathan	RFID readers	
Nakabuubi Hilda	diagram.json	https://youtu.be/EF6LE0LP7RI
Nyonyozi Maria Lisa Loyce	Backend (MQTT bridge, REST API, database) and management dashboard	
Okoth Victor	LCD monitor	

TABLE OF CONTENTS

TASKS COMPLETED BY EACH MEMBER 2
LIST OF TABLES 5
LIST OF FIGURES 6
ABSTRACT 7
1. INTRODUCTION 8
1.1 Background and Problem Statement 8
1.2 Objectives 8
1.3 Scope 8
1.4 Definitions and Acronyms 9
2. SYSTEM REQUIREMENTS 10
2.1 Functional Requirements 10
2.2 Non-Functional Requirements 10
3. USE CASE ANALYSIS 12
3.1 Actors 12

3.2 System Use Case Diagram 12
3.3 Controller, Bridge and API Use Case Diagram 13
3.4 Use Case Summary 14
3.5 Use Case Specifications 15
4. SYSTEM ARCHITECTURE 20
4.1 Overview 20
4.2 Design Decision: One ESP32 Controller 20
4.3 End-to-End System Architecture 20
4.4 Network Design 21
4.5 Architectural Modules 22
4.6 Concurrency Model 23
4.7 Data Flow for an Access Request 24
4.8 Communication Interfaces 25
4.9 Server-Side Components 25
5. HARDWARE COMPONENTS 28
5.1 Component List 28
5.2 Pin Assignment 28
5.3 Hardware Design Notes 29
6. SOFTWARE DESIGN 30
6.1 Development Environment 30
6.2 Start-up Sequence 30
6.3 Access Decision Logic 30
6.4 RFID Driver 31
6.5 Door Control 32
6.6 Storage and Persistence 32
6.7 Communication 32
6.8 Room Automation 33
6.9 Serial Console 34
6.10 LCD Driver 34
6.11 Time Synchronisation 35
7. DATABASE DESIGN 36
7.1 Choice of Database Engine 36
7.2 Entity-Relationship Diagram 36
7.3 Tables 37
7.4 Integrity Rules, Indexes and Views 38
7.5 Initial Data 39
8. REST API AND HOW TO ACCESS IT 40

8.1 General Conventions 40
8.2 Security Model 40
8.3 Endpoint Summary 40
8.4 Starting the API 42
8.5 Accessing the API with Swagger UI 43
8.6 Accessing the API from Other Clients 44
8.7 Demonstration Credentials 45
9. MANAGEMENT DASHBOARD 47
9.1 Accessing the Dashboard and Logging In 47
9.2 Roles and Permissions 47
9.3 Overview Tab 48
9.4 Schedule Tab 49
9.5 Users Tab 50
9.6 Reports Tab 50
9.7 Audit Log Tab 51
9.8 Database Tab 51
10. IMPLEMENTED FEATURES 53
11. SYSTEM OPERATION 54
11.1 Normal Operation 54
11.2 Administrator Operation 54
11.3 Monitoring 54
11.4 Dashboard Operation 54
12. TESTING AND RESULTS 56
12.1 Automated Database, API and Bridge Tests 56
12.2 End-to-End Test over the MQTT Broker 57
12.3 Reference HTTP Controller and Virtual ESP32 58
13. LIMITATIONS AND FUTURE WORK 60
13.1 Known Limitations 60
13.2 Future Work 60
14. CONCLUSION 61
REFERENCES 61

LIST OF TABLES

TABLE I FUNCTIONAL REQUIREMENTS 10
TABLE II ACTORS OF THE SYSTEM 12
TABLE III SUMMARY OF USE CASES 14
TABLE IV USE CASE SPECIFICATION: UC01 ENTER BUILDING 15
TABLE V USE CASE SPECIFICATION: UC02 EXIT BUILDING 16
TABLE VI USE CASE SPECIFICATION: UC03 VERIFY ACCESS 16
TABLE VII USE CASE SPECIFICATION: UC05 CONTROL ROOM LIGHTING 17
TABLE VIII USE CASE SPECIFICATION: UC06 SWITCH LIGHTS OFF WHEN VACANT 17
TABLE IX USE CASE SPECIFICATION: UC07 VIEW OCCUPANCY 17
TABLE X USE CASE SPECIFICATION: UC11 CONFIGURE ACCESS SCHEDULE 18
TABLE XI USE CASE SPECIFICATION: UC12 LOCK DOWN THE ENTRANCE 18
TABLE XII USE CASE SPECIFICATION: UC15 RELAY MESSAGES TO THE DATABASE 19
TABLE XIII CONNECTIONS BETWEEN THE PARTS OF THE SYSTEM 22
TABLE XIV SOFTWARE MODULES AND RESPONSIBILITIES 23
TABLE XV FREERTOS TASKS 23
TABLE XVI INTERNAL AND EXTERNAL INTERFACES 25
TABLE XVII WHAT THE MQTT BRIDGE DOES WITH EACH MESSAGE 26
TABLE XVIII COMPONENT LIST 28
TABLE XIX GPIO ASSIGNMENT 28
TABLE XX ACCESS DECISION RULES 31
TABLE XXI MQTT TOPICS 33
TABLE XXII CONSOLE COMMANDS 34
TABLE XXIII TABLES OF THE CENTRAL DATABASE 38
TABLE XXIV INTEGRITY CONSTRAINTS, INDEXES AND VIEWS 38
TABLE XXV GENERAL API CONVENTIONS 40
TABLE XXVI SUMMARY OF API ENDPOINTS 41
TABLE XXVII SCRIPTS FOR INSTALLING, STARTING AND TESTING THE BACKEND 42
TABLE XXVIII DEMONSTRATION CREDENTIALS (CHANGE BEFORE A REAL DEPLOYMENT) 45
TABLE XXIX DASHBOARD AND API ROLES 48
TABLE XXX AUTOMATED TEST RESULTS 56
TABLE XXXI END-TO-END TEST OVER THE MQTT BROKER 57

LIST OF FIGURES

Fig. 1. System use case diagram of the CoCIS Smart Control and Monitoring System. 13
Fig. 2. Use case diagram of the entrance ESP32, the MQTT bridge and the REST API. 14
Fig. 3. End-to-end architecture of the CoCIS Smart Control and Monitoring System. 21
Fig. 4. Network view: where each part runs and how it connects. 22
Fig. 5. Sequence of a card scan, from the reader to the database and the dashboard. 25
Fig. 6. Entity-relationship diagram of the cocis.db database. 37
Fig. 7. Swagger UI at /docs: instructions, demonstration data and the device endpoints. 43
Fig. 8. Swagger UI Authorize dialog: device key and administrator login. 44
Fig. 9. Executing POST /device/access/check in Swagger UI: the request, the curl command and the GRANTED response. 44
Fig. 10. Dashboard login page. 47
Fig. 11. Dashboard Overview tab: occupancy, lecture rooms, controller status and recent access events. 49
Fig. 12. Dashboard Schedule tab: opening hours per day. 50
Fig. 13. Dashboard Users tab: registering and deactivating card holders. 50
Fig. 14. Dashboard Reports tab: daily totals, peak occupancy and lighting energy. 51
Fig. 15. Dashboard Audit log tab. 51
Fig. 16. Database tab: access events stored from the entrance ESP32 over MQTT. 52
Fig. 17. Database tab: room status changes (MOTION and NO_MOTION) stored from the entrance ESP32. 52
Fig. 18. MQTT bridge window: messages from the entrance ESP32 stored in the database. 58
Fig. 19. Reference HTTP controller in Wokwi: start-up, offline decisions, room count and status. 59
Fig. 20. Virtual ESP32 demonstration (demo.bat), steps 2 to 7. 59

ABSTRACT

This report describes the design, implementation and operation of the CoCIS Entrance Node, an embedded access-control system built around an ESP32 microcontroller. The system reads RFID cards at separate entry and exit readers, checks the validity of the card and decides whether to open each door using a card whitelist, an opening-hours schedule and an anti-passback rule. It then drives servo-actuated doors with LED, buzzer and LCD feedback. It tracks building occupancy, keeps a persistent audit log on an SD card, stores configuration in non-volatile memory, and publishes events and state to an MQTT broker as JSON. Two PIR motion sensors automate the lighting of two rooms. An administrator can inspect and configure the device through a serial console. The firmware is written in C on ESP-IDF using FreeRTOS and is demonstrated in the Wokwi simulator. A management dashboard, fed by an MQTT bridge, a REST API and an SQLite database, lets staff monitor occupancy, lecture rooms and the access history from a web browser.

***Index Terms**: Access control, ESP32, ESP-IDF, FreeRTOS, MQTT, MFRC522, RFID, Internet of Things, REST API, dashboard, Wokwi.*

1. INTRODUCTION

1.1 Background and Problem Statement

CoCIS at Makerere University houses valuable equipment, such as the laboratory computers, that requires reliable security measures to prevent unauthorised access and protect institutional resources. In addition to security concerns, the college faces high electricity costs, partly because lights are left on in unoccupied rooms. Traditional key-based systems also provide no record of who entered and when. We therefore proposed using low-cost ESP32 microcontrollers with integrated Wi-Fi to build networked access-control nodes that authenticate users with RFID cards and log every attempt. This provides controlled entry to the building, protects people and equipment by showing the occupancy at any given moment, supports evacuation planning, and saves electricity by switching lights off when a room is unoccupied.

1.2 Objectives

- Authenticate users at entry and exit readers using RFID tags.
- Enable actuation of the entry and exit doors and give immediate visual, audible and textual feedback on the LCD screen.
- Enforce an opening-hours schedule and an anti-passback rule.
- Automate room lighting with PIR motion sensors and report room status to the entrance node via MQTT.
- Maintain a live occupancy count and store state in non-volatile memory to survive power interruptions.
- Keep a timestamped audit log on an SD card for persistence and publish events to an MQTT broker.
- Provide a secured serial console for inspection and configuration.
- Demonstrate the complete system using the Wokwi simulator.

1.3 Scope

This pilot project initially covers one entrance node with two doors (entry and exit) and two monitored lecture rooms. It covers the embedded firmware and the simulated hardware, together with a management dashboard and the supporting backend (an MQTT bridge, a REST API and an SQLite database) that consume the node's MQTT messages.

1.4 Definitions and Acronyms

Acronym	Definition
MQTT	Message Queuing Telemetry Transport
RFID tag	Radio Frequency Identification tag
ESP32	The microcontroller used in the system
ESP-IDF	Espressif IoT Development Framework used to build the firmware
FreeRTOS	Free Real-Time Operating System
LED	Light Emitting Diode
LCD Screen	Liquid Crystal Display
PIR sensor	Passive Infrared motion sensor
API	Application Programming Interface
REST	Representational State Transfer
JSON	JavaScript Object Notation
NVS	Non-Volatile Storage (flash memory of the ESP32)
UML	Unified Modeling Language

2. SYSTEM REQUIREMENTS

2.1 Functional Requirements

TABLE I

FUNCTIONAL REQUIREMENTS

ID	Requirement
FR1	The system shall read the unique identifier (UID) of an RFID card presented to either entry or exit reader.
FR2	The system shall grant access only to UIDs on the authorised list.
FR3	The system shall deny entry outside the configured opening hours; exit shall always be permitted if the person is inside.
FR4	The system shall deny a second entry by a card already inside and deny exit by a card not inside.
FR5	The system shall open the corresponding door for a fixed time after a granted request.
FR6	The system shall give feedback through a green or red LED, a buzzer and a 16x2 LCD.
FR7	The system shall maintain an occupancy count and the list of cards inside.
FR8	The system shall persist the schedule and the inside list in non-volatile memory.
FR9	The system shall append every access attempt to a CSV log on an SD card.
FR10	The system shall publish occupancy, access events, schedule and room status to an MQTT broker.
FR11	The system shall switch each room light on when motion is detected and off after five seconds without motion.
FR12	The system shall provide a serial console in which schedule changes require an authenticated administrator session that expires automatically.
FR13	The system shall obtain the current time from a network time server.

2.2 Non-Functional Requirements

- Responsiveness: a card presented to a reader shall be processed within a fraction of a second (readers are polled every 150 ms).

- Reliability: failure of the SD card or the MQTT connection shall not prevent doors from operating.
- Thread safety: all shared data shall be protected against concurrent access by multiple tasks.
- Maintainability: pins, network settings, topics, authorised cards and limits shall be centralised in one configuration header.
- Testability: the whole system shall run unmodified in the Wokwi simulator.

3. USE CASE ANALYSIS

Before designing the hardware and software, the group identified who uses the system and what each user needs it to do. These needs are captured below as UML use cases. Each use case is traced to the firmware module or backend component that implements it, so every part of the design can be explained by the need it serves.

3.1 Actors

Table II lists the actors. People interact with the system directly; the timer and the two external systems take part in use cases without a person being involved.

TABLE II

ACTORS OF THE SYSTEM

Actor	Type	Description
Building User	Person	Any student, staff member, lecturer or visitor who holds an RFID card and passes through the main entrance.
Room Occupant	Person	Anyone in Lecture Room 1 or 2. Detected by the room's PIR motion sensor; needs no card.
Management Viewer	Person (role VIEWER)	College management or reception staff who monitor occupancy, logs and reports on the dashboard.
Security Officer	Person (role SECURITY)	Everything a Viewer can do, plus emergency mode, lockdown of the entrance and light overrides.
System Administrator	Person (role SUPER_ADMIN; admin PIN on the ESP32)	Everything a Security Officer can do, plus managing authorised cards, the access schedule and settings.
Time	Timer	The five-second motion timeout that switches a room's lights off.
Entrance ESP32	External system	The controller: decides every scan and publishes events over MQTT.
MQTT Bridge	External system	Program on the server that relays the ESP32's messages to the REST API and forwards lockdown commands back.

3.2 System Use Case Diagram

Fig. 1 shows the use cases of the whole system. Use cases on the left are carried out by people in the building; those on the right are management functions. Dashed arrows show «include» relationships (a step that is always performed, such as verifying a card) and

«extend» relationships (optional behaviour, such as a manual lighting override). The hollow-headed arrows between the staff actors show that each role inherits the abilities of the one below it.

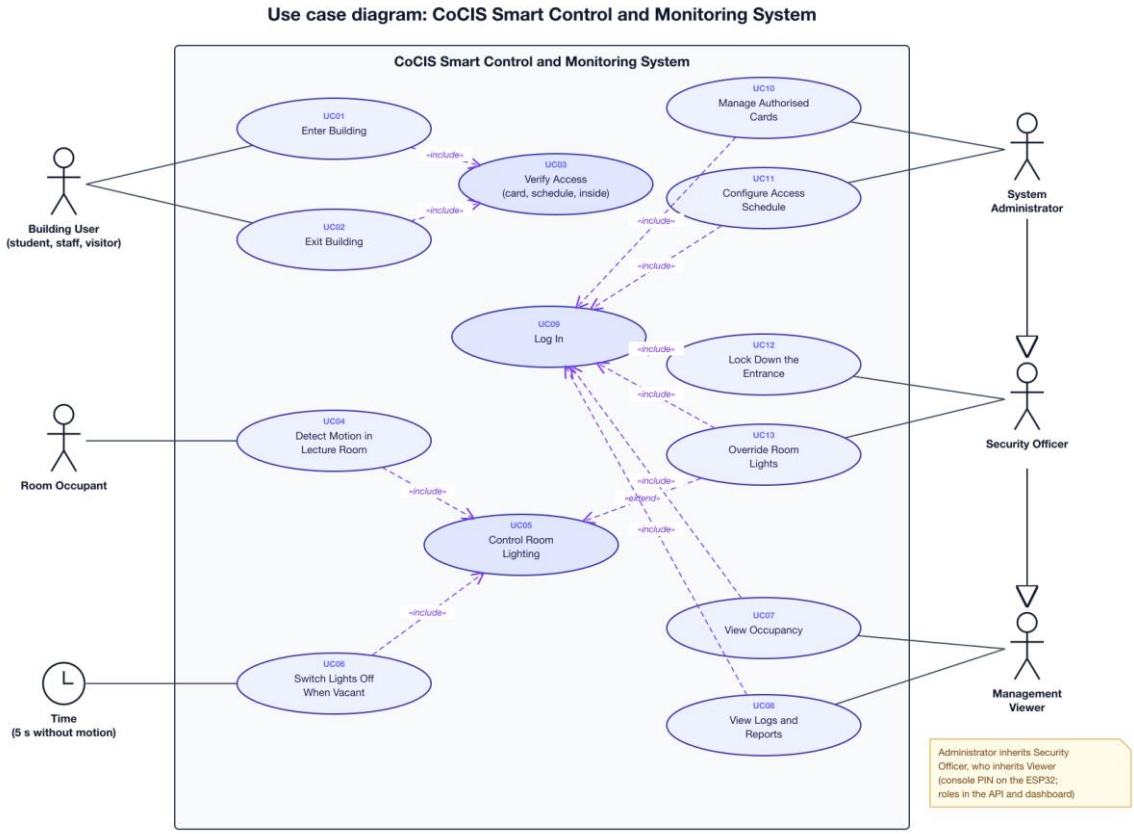


Fig. 1. System use case diagram of the CoCIS Smart Control and Monitoring System.

3.3 Controller, Bridge and API Use Case Diagram

Fig. 2 shows how the controller's data reaches the database. The ESP32 never calls the API itself: it publishes to the MQTT broker, and the bridge makes the API requests. Every request the bridge makes is authenticated with the device key, and every event is stored exactly once, however many times it is delivered.

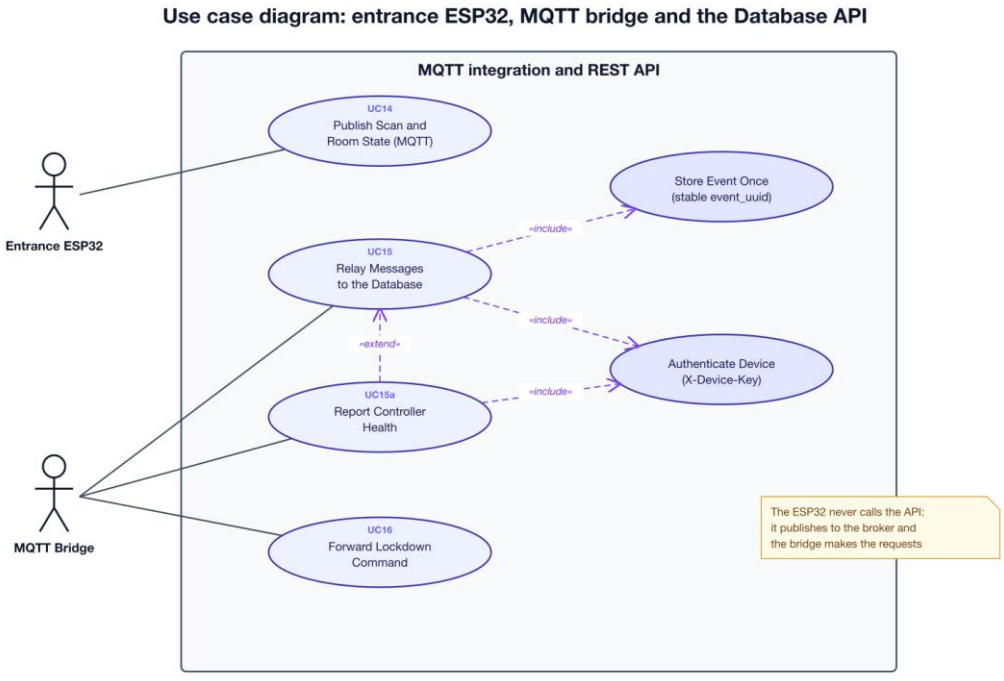


Fig. 2. Use case diagram of the entrance ESP32, the MQTT bridge and the REST API.

3.4 Use Case Summary

Table III summarises every use case with its primary actor and the part of the system that implements it.

TABLE III

SUMMARY OF USE CASES

ID	Use case	Primary actor	Implemented by
UC01	Enter Building	Building User	rfid.c, state.c, door.c; bridge; access_events, building_visits
UC02	Exit Building	Building User	rfid.c, state.c, door.c; bridge; access_events, building_visits
UC03	Verify Access (card, schedule, inside)	Included by UC01, UC02	state_evaluate() in state.c
UC04	Detect Motion in Lecture Room	Room Occupant	rooms.c (room_lights task)
UC05	Control Room Lighting	Included by UC04, UC06	rooms.c, comms.c; bridge; room_status_logs
UC06	Switch Lights Off When Vacant	Time	HOLD_TICKS in rooms.c
UC07	View Occupancy	Management Viewer	Dashboard Overview; GET /occupancy; console status, who
UC08	View Logs and Reports	Management Viewer	Dashboard Reports, Audit log and Database tabs

ID	Use case	Primary actor	Implemented by
UC09	Log In	Any staff actor	POST /auth/login; console login
UC10	Manage Authorised Cards	System Administrator	Console user add/remove; dashboard Users tab
UC11	Configure Access Schedule	System Administrator	Console schedule set; dashboard Schedule tab
UC12	Lock Down the Entrance	Security Officer	PUT /settings/lockdown; bridge; comms.c
UC13	Override Room Lights	Security Officer	Console lights command; dashboard light mode
UC14	Publish Scan and Room State	Entrance ESP32	comms.c (MQTT topics)
UC15	Relay Messages to the Database	MQTT Bridge	bridge/mqtt_bridge.py; POST /device/relay
UC15a	Report Controller Health	MQTT Bridge	POST /device/heartbeat; devices table
UC16	Forward Lockdown Command	MQTT Bridge	Bridge publishes to cocis/cmd/entrance

3.5 Use Case Specifications

The main use cases are specified below. The main flow is the normal sequence of steps; the alternative flows cover errors and exceptions (step 3a is an alternative to step 3).

TABLE IV

USE CASE SPECIFICATION: UC01 ENTER BUILDING

Primary actor	Building User
Goal	Enter CoCIS through the entry lane of the main entrance.
Preconditions	The user holds an RFID card. The ESP32 is running.
Main flow	1. The user taps the card on the entry reader (MFRC522). 2. The RFID task reads the four-byte UID and the time. 3. The ESP32 verifies access (UC03). 4. The ESP32 opens the entry door servo for three seconds, lights the green LED, beeps briefly and shows ACCESS GRANTED and the UID on the LCD. 5. The ESP32 adds the card to its inside list, appends the scan to access.csv on the SD card and publishes it over MQTT (UC14). 6. The bridge stores the scan and opens a visit in the database (UC15).

Alternative flows	3a. Access is denied: the door stays shut, the red LED lights, the buzzer sounds for 0.5 s and the LCD shows ACCESS DENIED. The refused scan is still logged and published. 5a. The broker cannot be reached: the scan is still decided, logged on the SD card and shown on the LCD, but it does not reach the database.
Postconditions	A row exists in access_events. If access was granted, the user has an open row in building_visits.
Implemented by	rfid.c, state.c, door.c; bridge POST /device/relay; tables access_events and building_visits

TABLE V

USE CASE SPECIFICATION: UC02 EXIT BUILDING

Primary actor	Building User
Goal	Leave CoCIS through the exit lane.
Preconditions	The user is recorded as inside the building.
Main flow	1. The user taps the card on the exit reader. 2. The ESP32 verifies access (UC03). 3. The ESP32 opens the exit door for three seconds and removes the card from the inside list. 4. The scan is logged, published and stored; the visit receives its time out.
Alternative flows	2a. The card is not recorded as inside: the exit is refused (DENIED_NOT_IN). 2b. Outside the opening hours: the exit door still opens, so nobody is trapped inside, but the result is OK_AFTER_HOURS and it is stored as such so that security can follow up.
Postconditions	The visit has a time out, and the person no longer appears in the people-inside list.
Implemented by	state_evaluate() in state.c; bridge POST /device/relay; tables access_events and building_visits

TABLE VI

USE CASE SPECIFICATION: UC03 VERIFY ACCESS

Primary actor	None (included by UC01 and UC02)
Goal	Decide on the ESP32 whether a scanned card may pass, and return a result code.
Preconditions	A card UID, the reader direction and the time have been read.
Main flow	1. Take the state mutex. 2. If lockdown is active, refuse. 3. Check that the card is in the authorised list. 4. For an entry, check that the time is inside the opening window and the card is not already inside. 5. For an exit, check that the card is inside. 6. Return the result code (Table XX), update the inside list and release the mutex.

Alternative flows	1a. Any check fails: the matching DENIED code is returned and nothing else changes.
Postconditions	The decision is made locally, so the doors work even without a network.
Implemented by	state_evaluate() in state.c

TABLE VII

USE CASE SPECIFICATION: UC05 CONTROL ROOM LIGHTING

Primary actor	None (included by UC04 and UC06; extended by UC13)
Goal	Keep the lights on only while a lecture room is in use.
Preconditions	The room's motion state and any override are known.
Main flow	1. If an override is set (on or off), use it; otherwise use the motion state. 2. When the state changes, switch the light output. 3. Publish the new state to cocis/state/roomN as a retained message. 4. The bridge stores it in rooms and room_status_logs with the trigger MOTION or NO_MOTION.
Alternative flows	3a. No broker: the light still switches; only the database is not updated.
Postconditions	The light matches the room's use, and the change is logged for the energy report.
Implemented by	rooms.c, comms.c; bridge; room_status_logs; GET /reports/daily

TABLE VIII

USE CASE SPECIFICATION: UC06 SWITCH LIGHTS OFF WHEN VACANT

Primary actor	Time
Goal	Save energy when a room is no longer in use.
Preconditions	The room is in automatic mode and its light is on.
Main flow	1. Five seconds pass with no motion. 2. The room becomes vacant and the light switches off (UC05). 3. The change is published and stored with the trigger NO_MOTION.
Alternative flows	1a. Motion returns before the five seconds end: the timer restarts and the light stays on.
Postconditions	The light is off and the room shows as vacant on the dashboard.
Implemented by	HOLD_TICKS in rooms.c

TABLE IX

USE CASE SPECIFICATION: UC07 VIEW OCCUPANCY

Primary actor	Management Viewer (and, by inheritance, Security Officer and System Administrator)
Goal	See who is in the building now and the state of each lecture room.

Preconditions	Logged in to the dashboard (UC09), or at the ESP32 console.
Main flow	1. The viewer opens the dashboard Overview tab (or types status, who or occupancy at the console). 2. The system shows everyone inside with their time in, the total, and each room's state and light.
Alternative flows	1a. The login token has expired: the API answers 401 and the viewer logs in again.
Postconditions	None (read only).
Implemented by	GET /occupancy; view v_people_inside; table rooms; console status command

TABLE X

USE CASE SPECIFICATION: UC11 CONFIGURE ACCESS SCHEDULE

Primary actor	System Administrator
Goal	Set the daily hours during which the building may be entered.
Preconditions	An admin session on the ESP32 (UC09).
Main flow	1. The administrator types schedule set HH:MM HH:MM. 2. The ESP32 saves the window to NVS and enforces it from the next scan. 3. It publishes the schedule as a retained message; the bridge copies it to access_schedule (PUT /schedule), so the dashboard shows the hours in use.
Alternative flows	1a. No admin session: the command is refused. 3a. A window that crosses midnight is enforced on the ESP32 but not copied, because the database stores one window per day.
Postconditions	The ESP32 and the database show the same opening hours.
Implemented by	console.c, state.c, comms.c; bridge

TABLE XI

USE CASE SPECIFICATION: UC12 LOCK DOWN THE ENTRANCE

Primary actor	Security Officer
Goal	Refuse every card at once during a security threat.
Preconditions	Logged in as SECURITY or SUPER_ADMIN.
Main flow	1. The officer switches lockdown on (PUT /settings/lockdown) with a reason; the change is audited. 2. Within two seconds the bridge publishes LOCKDOWN to cocis/cmd/entrance as a retained message. 3. The ESP32 sets its lockdown flag and refuses every scan. 4. UNLOCK ends the lockdown in the same way.
Alternative flows	3a. In the current firmware the topic check compares 19 characters instead of 18, so the command is ignored until that value is corrected (see Known Limitations).
Postconditions	The doors follow the lockdown state, and every change is audited.

Implemented by	PUT /settings/lockdown; bridge watch_lockdown(); MQTT handler in comms.c

TABLE XII

USE CASE SPECIFICATION: UC15 RELAY MESSAGES TO THE DATABASE

Primary actor	MQTT Bridge
Goal	Store every ESP32 message in the database exactly once.
Preconditions	The API is running and the bridge is subscribed to the controller's topics.
Main flow	1. The bridge receives a message and converts it (Table XVII). 2. It gives the event a stable event_uuid built from the card, the direction and the ESP32 time. 3. It sends POST /api/v1/device/relay with its device key. 4. The API stores the event in one transaction and reports it as accepted or as a duplicate. 5. The bridge also sends a heartbeat (UC15a), so the dashboard shows the controller as online.
Alternative flows	1a. A retained message older than ten minutes is ignored. 3a. The API is down: the error is shown and the message is lost from the database, but the ESP32's SD log still has it.
Postconditions	The database matches the ESP32's events, and the dashboard updates within three seconds.
Implemented by	bridge/mqtt_bridge.py; POST /device/relay; POST /device/heartbeat

4. SYSTEM ARCHITECTURE

4.1 Overview

The system follows a layered, task-based architecture running on a single ESP32. At the lowest layer, hardware drivers (RFID, servo, LCD, SD card, GPIO) interact with the peripherals. Above them sit the shared-state and storage services, and at the top an orchestration layer (the RFID workflow, the room task and the console) implements behaviour and publishes information through the communications module. Each module exposes a small header interface, and only the state module owns shared data, which it protects with a mutex.

The sections that follow describe the end-to-end architecture of the complete system and its network design, the firmware architecture on the ESP32 (Sections 4.5 to 4.8) and the server-side components that carry the controller's data to the management dashboard (Section 4.9).

4.2 Design Decision: One ESP32 Controller

The coursework brief describes three control terminals (Lecture Room 1, Lecture Room 2 and the main entrance), each of which may use its own ESP32. The free version of the Wokwi simulator supports only one microcontroller per project, and the course lecturer confirmed that groups should use one. The design therefore uses one ESP32 that serves all three terminals.

The terminals still exist logically: the entrance is handled by the RFID and door tasks, and each lecture room by the room task. If CoCIS later moves to one ESP32 per terminal, each board can publish to the same MQTT topics, and the bridge, API and database stay exactly the same.

4.3 End-to-End System Architecture

Fig. 3 shows the final architecture in four layers, read from left to right:

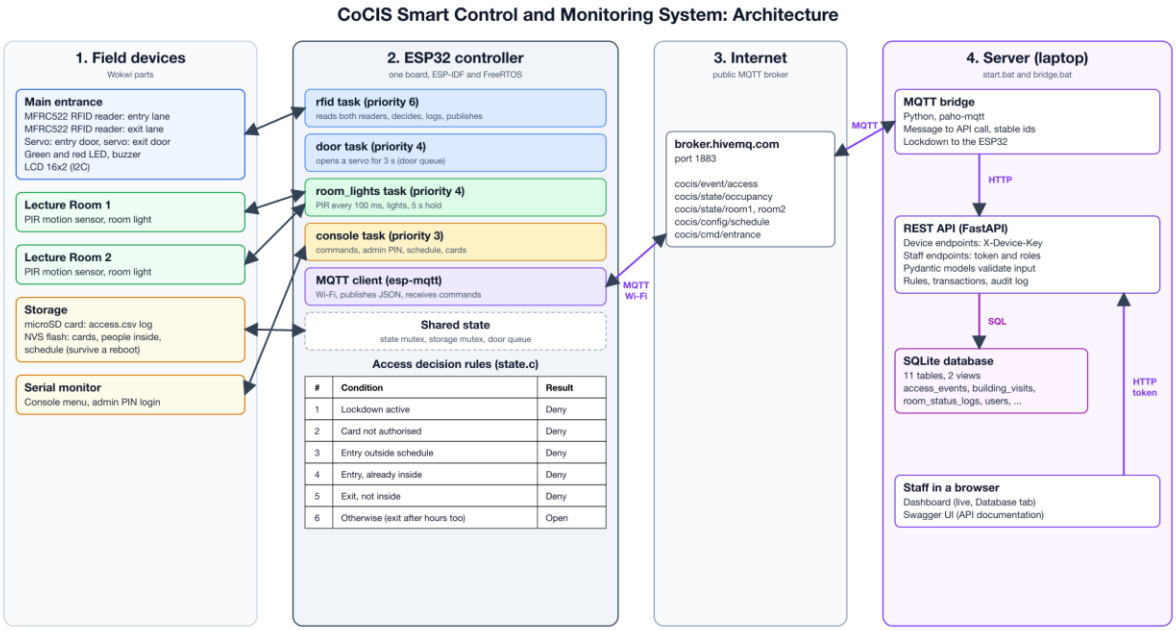


Fig. 3. End-to-end architecture of the CoCIS Smart Control and Monitoring System.

- **Layer 1, field devices:** at the main entrance, an MFRC522 RFID reader and a door servo for each lane, a green and a red LED, a buzzer and a 16x2 LCD; in each lecture room, a PIR motion sensor and the room light; for storage and administration, a microSD card, the ESP32's NVS flash and the serial monitor.
- **Layer 2, the ESP32 controller:** one board running the FreeRTOS tasks described in Section 4.6. Every access decision is made here, so the doors keep working without a network.
- **Layer 3, the MQTT broker:** the public broker broker.hivemq.com on port 1883 passes messages between the ESP32 and the server. Neither side connects to the other directly.
- **Layer 4, the server:** the MQTT bridge, the REST API and the SQLite database run on a laptop or college server, and staff use the management dashboard and Swagger UI in a web browser.

4.4 Network Design

Fig. 4 answers a common question: does the ESP32 need to be on the same network as the database? It does not. The ESP32 (in Wokwi, on the Wokwi-GUEST Wi-Fi) and the server each open an outgoing connection to the MQTT broker on the internet, and the broker passes the messages between them. Because nobody needs to connect into the laptop, no router

settings or port forwarding are needed, and the database is never exposed to the internet. Table XIII lists each connection.

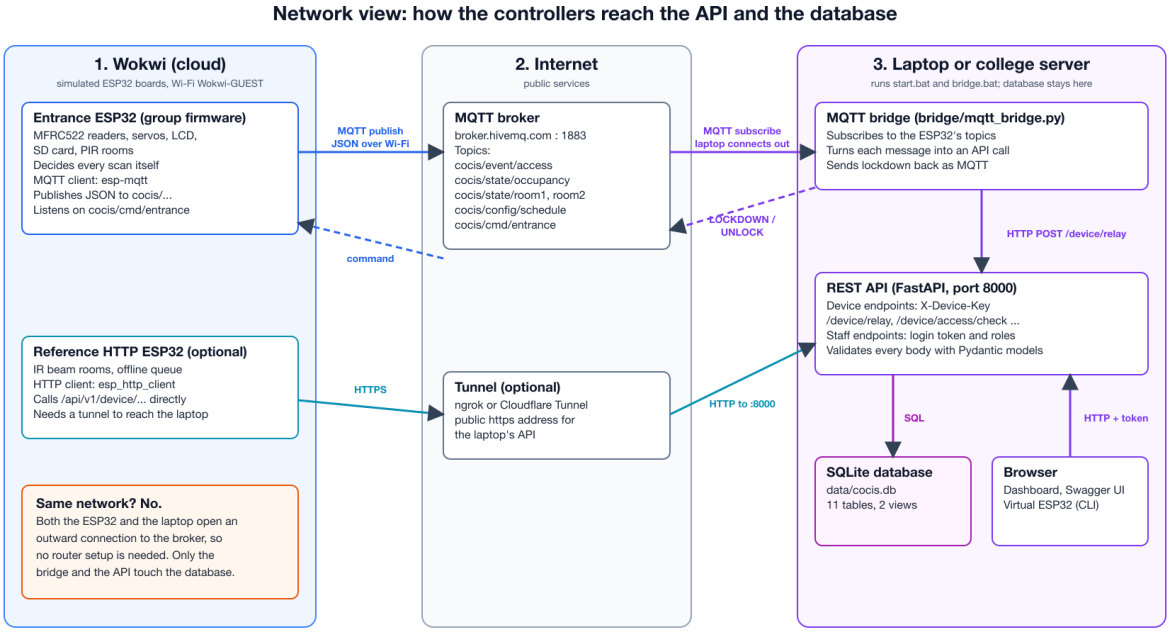


Fig. 4. Network view: where each part runs and how it connects.

TABLE XIII

CONNECTIONS BETWEEN THE PARTS OF THE SYSTEM

Link	**Protocol**	**Who connects**	**Network needed**
ESP32 to broker	MQTT 3.1.1 over TCP, port 1883, JSON payloads	ESP32 connects out	Internet (Wokwi-GUEST Wi-Fi in the simulator)
Broker to bridge	MQTT subscription	Bridge connects out	Internet on the server
Bridge to REST API	HTTP, JSON, X-Device-Key header	Bridge calls 127.0.0.1:8000	Same computer
API to database	SQL through Python sqlite3	Inside the API process	Same computer (one file)
Dashboard to API	HTTP, JSON, bearer token	The web browser	Same computer, or the college network
Bridge to ESP32 (lockdown)	MQTT publish to cocis/cmd/entrance	Bridge publishes; ESP32 subscribed	Internet

For a real deployment the broker would be a private one, for example Mosquitto on the college server with a username, a password and TLS on port 8883. Only the broker address in the firmware's config.h and the bridge's broker option would change.

4.5 Architectural Modules

TABLE XIV

SOFTWARE MODULES AND RESPONSIBILITIES

Module	**Files**	**Responsibility**
Entry point	main.c	Creates the SPI bus, initialises all modules in order, starts the tasks, sets the time zone and starts SNTP.
Configuration	config.h	Pin numbers, Wi-Fi and MQTT settings, topic names, authorised card list, limits and timings.
Shared state	state.c, state.h	Owns occupancy, inside list, schedule, admin session and room flags; evaluates access requests.
RFID	rfid.c, rfid.h	Register-level MFRC522 driver for two readers and the complete card-tap workflow.
Door	door.c, door.h	PWM servo control through a request queue and a dedicated task.
Storage	storage.c, storage.h	NVS storage for schedule and inside list; SD-card CSV audit log.
Communications	comms.c, comms.h	Wi-Fi station setup, MQTT client, JSON publishing and subscription handling.
Rooms	rooms.c, rooms.h	PIR polling, light control, room status updates and publishing.
Console	console.c, console.h	UART command-line interface with admin authentication.
Display	lcd.c, lcd.h	HD44780 16x2 LCD driver over I2C and the idle and result screens.

4.6 Concurrency Model

The firmware uses FreeRTOS tasks so that slow actions, such as holding a door open for three seconds, never block card reading or room monitoring. Table XV lists the tasks. Producers communicate with the door task through a queue, and all access to the shared state goes through mutex-protected functions.

TABLE XV

FREERTOS TASKS

Task	**Priority**	**Stack (bytes)**	**Function**
main (app_main)	Default	8192	Initialisation, then returns; other tasks continue.

Task	Priority	Stack (bytes)	Function
rfid	6	8192	Polls the entry and exit readers every 150 ms and runs the access workflow.
door	4	2048	Blocks on a queue; moves a servo to the open position, waits, and closes.
room_lights	4	4096	Reads both PIR sensors every 100 ms and controls the lights.
console	3	4096	Reads typed characters from UART0 and executes commands.
Library tasks	Library-defined	Internal	Wi-Fi, TCP/IP, MQTT client and SNTP operate in tasks created by ESP-IDF.

4.7 Data Flow for an Access Request

1. The RFID task reads a card UID from a reader and converts it to a 32-bit number.
2. Repeated reads of the same card within 1.5 seconds are ignored.
3. The state module evaluates the request and returns one of six results.
4. If the result is a grant, a request is placed on the door queue and the door task moves the servo.
5. LEDs, the buzzer and the LCD show the result.
6. The event is appended to the SD log, published to MQTT, and the occupancy is published.
7. The inside list and schedule are saved to non-volatile memory.
8. After three seconds the LCD returns to the idle screen.

Fig. 5 follows the same scan beyond the controller. After the door has opened, the event travels through the MQTT broker and the bridge to the REST API, which stores it in the database in one transaction, and the dashboard shows it within three seconds.

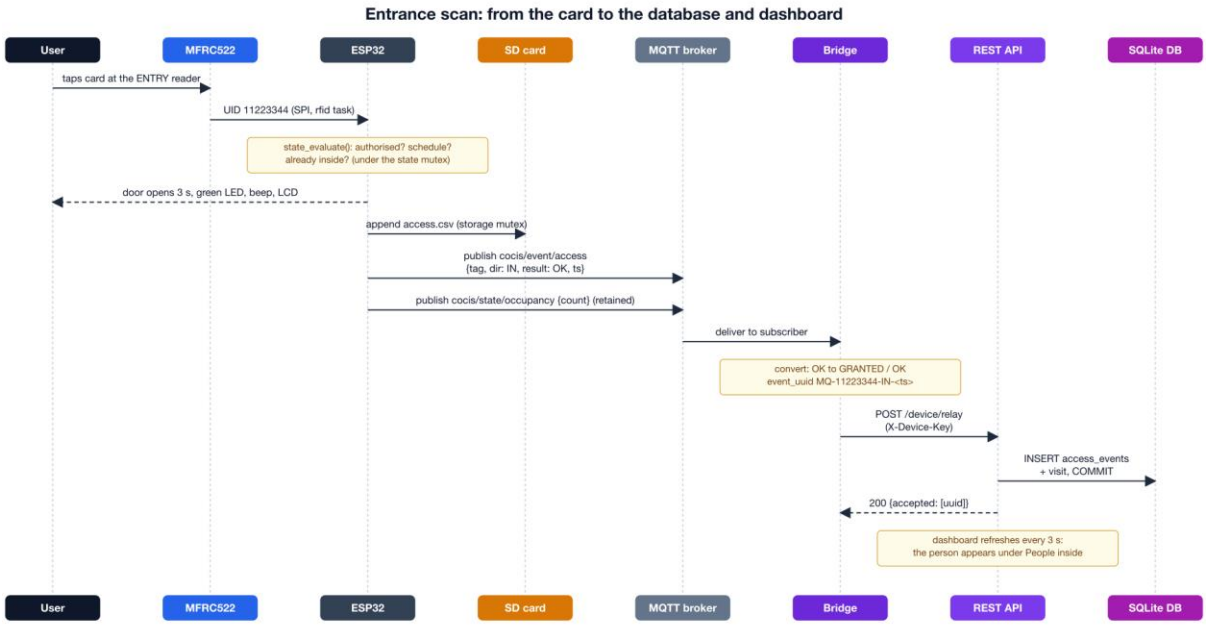


Fig. 5. Sequence of a card scan, from the reader to the database and the dashboard.

4.8 Communication Interfaces

TABLE XVI

INTERNAL AND EXTERNAL INTERFACES

Interface	**Used for**	**Settings**
SPI (SPI2 host)	Two MFRC522 readers and the microSD card on one shared bus	SCK 18, MISO 19, MOSI 23; separate chip-select per device; readers at 1 MHz, mode 0
I2C (port 0)	16x2 LCD through a PCF8574 backpack	SDA 21, SCL 22, address 0x27, 100 kHz
LEDC PWM	Two servo motors	50 Hz, 16-bit resolution
UART0	Serial console	115200 baud, 8N1
Wi-Fi (station mode)	Network access	SSID Wokwi-GUEST, open network
MQTT over TCP	Publishing and subscribing	Broker broker.hivemq.com, port 1883, keep-alive 30 s

4.9 Server-Side Components

The server side gives management a permanent, searchable record of everything the controller does. It consists of four parts:

- **MQTT bridge** (bridge/mqtt_bridge.py): subscribes to the controller's topics with the Eclipse Paho client, converts each message and calls the REST API with a device key. It needs no change to the firmware.
- **REST API** (FastAPI): device endpoints authenticated with the X-Device-Key header, and staff endpoints authenticated with a login token and three roles (VIEWER, SECURITY and SUPER_ADMIN). Every request body is validated, every write runs in a database transaction, and every change is recorded in an audit log. Swagger UI at /docs documents and tests every endpoint.
- **SQLite database:** eleven tables and two views, including users, rooms, access_events, building_visits, room_status_logs, access_schedule, devices and audit_log.
- **Management dashboard:** the web interface for staff, described in Section 9.

Table XVII shows what the bridge does with each message from the controller. Three safeguards keep the database correct: a stable event_uuid means that a message delivered twice is stored once; retained messages older than ten minutes, left over from earlier runs, are ignored; and before SNTP has set the ESP32's clock, the bridge uses its own time.

TABLE XVII

WHAT THE MQTT BRIDGE DOES WITH EACH MESSAGE

ESP32 message	Bridge action	API request	Stored in
Scan on cocis/event/access	Convert the result code and build a stable event_uuid	POST /device/relay	access_events, building_visits
Room state on cocis/state/roomN	Occupied becomes count 1 and trigger MOTION; vacant becomes 0 and NO_MOTION	POST /device/relay	rooms, room_status_logs
Count on cocis/state/occupancy	Compare with the database	GET /occupancy	Shown in the bridge window
Window on cocis/config/schedule	Copy to all seven days if different	PUT /schedule	access_schedule
First granted scan of a new card	Register the card	POST /users	users
Any message	Mark the controller as online	POST /device/heartbeat	devices
Lockdown changed on the server	Publish LOCKDOWN or UNLOCK (retained)	GET /occupancy every 2 s	Reaches the ESP32

5. HARDWARE COMPONENTS

5.1 Component List

TABLE XVIII

COMPONENT LIST

Component	**Qty**	**Role in the system**
ESP32 DevKit-C v4 (4 MB flash)	1	Runs all firmware.
MFRC522 RFID reader (13.56 MHz)	2	Reads the card UID at the entry door or the exit door over SPI.
RFID tag	As required	Uniquely identifies a user. The simulation uses UID 11:22:33:44.
microSD card	1	Stores the access.csv audit log (SPI).
Servo motor	2	Opens and closes the entry door and exit door according to the PWM signal it receives.
Green LED	1	Access granted indicator.
Red LED	1	Access denied indicator.
Buzzer	1	Provides audible feedback.
PIR motion sensor	2	Detects presence in Rooms 1 and 2.
Yellow LED	2	Represent the lights for Rooms 1 and 2.
LCD 1602 with I2C backpack	1	Displays the welcome screen, occupancy number and access results.

5.2 Pin Assignment

TABLE XIX

GPIO ASSIGNMENT

GPIO	**Signal**	**Connected device**
18	SPI SCK	Both RFID readers and SD card
19	SPI MISO	Both RFID readers and SD card
23	SPI MOSI	Both RFID readers and SD card
5	Chip select	Entry RFID reader

GPIO	Signal	Connected device
4	Reset	Entry RFID reader
15	Chip select	Exit RFID reader
2	Reset	Exit RFID reader
16	Chip select	SD card
13	PWM	Entry door servo
33	PWM	Exit door servo
12	Output	Green LED
14	Output	Red LED
27	Output	Buzzer
34	Input	PIR sensor, Room 1
26	Output	Light LED, Room 1
35	Input	PIR sensor, Room 2
25	Output	Light LED, Room 2
21	I2C SDA	LCD
22	I2C SCL	LCD

5.3 Hardware Design Notes

All three SPI devices share one bus and are distinguished by their chip-select lines, so only one is active at a time. GPIO34 and GPIO35 are input-only pins on the ESP32 and are therefore well suited to the PIR sensors. In the simulation all modules and servos are powered from the 3.3 V rail. A physical build should power the servos from a separate 5 V supply with a common ground, and should review the use of the ESP32 boot-strapping pins GPIO2, GPIO12 and GPIO15, which are used here for the exit reader reset, the green LED and the exit reader chip select.

6. SOFTWARE DESIGN

6.1 Development Environment

The firmware is written in C using the ESP-IDF 5.x framework with the CMake and Ninja build system. The project is named “entrance”, and the build produces entrance.elf and entrance.bin which the Wokwi simulator loads. The MQTT client is the Espressif esp-mqtt component (version 1.1.0), obtained through the IDF Component Manager. Other components used are FreeRTOS, the SPI, GPIO, LEDC and UART drivers, the new I2C master driver, NVS flash, the FAT file system with SD-SPI, the Wi-Fi and network interface stack, SNTP and cJSON. Project-specific build settings are in sdkconfig.defaults: an 8 KB main-task stack, a 1000 Hz FreeRTOS tick, warning-level logging, a 4 MB flash with the single-application large partition table, and long file name support for FAT.

6.2 Start-up Sequence

The function app_main in main.c performs the following steps in order. It creates the shared SPI2 bus with the SCK, MISO and MOSI pins and a maximum transfer size of 4096 bytes. It initialises the LCD and shows the welcome screen. It then initialises the state, storage, door, RFID, rooms and communications modules and the console. Next it starts the RFID and room tasks, waits three seconds for Wi-Fi to associate, starts the MQTT client and the console task, sets the time zone to EAT-3 (UTC+3), and starts SNTP polling of pool.ntp.org. Finally it prints a banner inviting the user to type help.

6.3 Access Decision Logic

The function state_evaluate in state.c is the core of the system. It receives the card identifier, the reader direction and the current time. The time is first converted to minutes after midnight:

$$m = 60 \times hour + minute \qquad (1)$$

The schedule is stored as a start minute s and an end minute e. A time m is inside the allowed window when the following holds. If s equals e, access is always allowed. If s is less than e, the window is $s \le m < e$. Otherwise the window wraps midnight and is $m \ge s$ or $m <$

e. The decision is then made while holding the state mutex, according to Table XX.

TABLE XX

ACCESS DECISION RULES

Reader	Condition	Result	Effect on state
Either	Card not on the authorised list	DENIED_UNKNOWN	None
Entry	Outside the allowed window	DENIED_SCHEDULE	None
Entry	Card already inside	DENIED_ALREADY_IN	None
Entry	Otherwise	OK	Card added to inside list; occupancy increases by one
Exit	Card not inside	DENIED_NOT_IN	None
Exit	Inside, within window	OK	Card removed; occupancy decreases by one
Exit	Inside, outside window	OK_AFTER_HOURS	Card removed; occupancy decreases by one

Allowing exit at any time is a deliberate safety decision: nobody can be locked inside after hours.

6.4 RFID Driver

The MFRC522 driver is written directly against the chip’s registers. A register write is a two-byte SPI frame whose first byte is the address shifted left by one bit, and a read sets the most significant bit of the address byte. Initialisation pulses the hardware reset line, issues a soft reset, configures the internal timer, forces 100% ASK modulation, sets the CRC preset, and switches the antenna on through the TX control register.

A generic transceive routine clears the interrupt flags, flushes the FIFO, loads the bytes to send, starts the transmit-and-receive command and polls the interrupt register until a receive or idle flag is set or a timeout expires. It returns an error if the error register reports buffer overflow, collision, parity or protocol errors. Reading a UID follows the ISO/IEC 14443-A procedure: a request command (REQA, 0x26, sent with 7-bit framing) must return a two-byte answer, and then an anti-collision command (0x93, 0x20) must return five bytes, namely four UID bytes and a check byte. The check byte must equal the exclusive-OR of the four UID

bytes. The four UID bytes are combined into a 32-bit number in big-endian order, so that the UID 11:22:33:44 becomes 0x11223344.

A repeated scan of the same card within SCAN_COOLDOWN_MS (1500 ms) is ignored so a card resting on the reader is not counted repeatedly. Protocol errors are logged at most once every three seconds.

6.5 Door Control

Each servo expects a pulse every 20 ms whose width sets the angle. The LEDC timer runs at 50 Hz with 16-bit resolution, so the duty value for a pulse of width t is:

$$duty = (t / 20\ ms) \times 65536 \qquad (2)$$

A 1.0 ms pulse gives a duty of 3277 (closed position, about 0 degrees) and a 1.5 ms pulse gives 4915 (open position, about 90 degrees in the simulator). The function door_open places a request on a queue of four entries without waiting, so the caller is never blocked. The door task receives the request, moves the servo to the open duty, waits DOOR_OPEN_MS (3000 ms) and returns it to the closed duty.

6.6 Storage and Persistence

Non-volatile storage (NVS) holds the schedule start and end as two integers and the inside list as a single binary blob consisting of a count followed by up to sixteen 32-bit card identifiers. The loader validates the count before copying, which protects against a corrupted blob. The state module copies its data into local variables while holding the mutex and writes to flash after releasing it, so slow flash operations never block other tasks.

The SD card is mounted through the SD-SPI driver at /sdcard on the shared SPI2 bus with its own chip select. If the card has no valid file system it is formatted. The file access.csv is opened in append mode, and a header line is written when the file is empty. Each access attempt adds one row with the Unix epoch, an ISO 8601 local time, the tag in eight hexadecimal digits, the direction (IN or OUT) and the result text. The file is flushed after every row so that a power loss costs at most one record. If the card cannot be mounted the system logs a warning and continues without file logging.

6.7 Communication

The Wi-Fi module configures the station interface for an open network and connects. The MQTT client is created with a client identifier made from a fixed prefix and the six bytes of the device’s MAC address, which keeps identifiers unique when several nodes share a broker. Table XXI lists the topics. State topics are published as retained messages with quality of service 1, so a dashboard that connects later immediately receives the latest values. Access events are published with quality of service 0 and are not retained, because each event is a one-time fact.

TABLE XXI

MQTT TOPICS

Topic	**Direction**	**Flags**	**Payload (JSON)**
cocis/state/occupancy	Publish	Retained, QoS 1	{"count":N,"ts":epoch}
cocis/event/access	Publish	Not retained, QoS 0	{"tag":"11223344","dir":"IN","result":"OK","ts":epoch}
cocis/config/schedule	Publish	Retained, QoS 1	{"start":"07:00","end":"21:00","version":epoch}
cocis/state/room1, cocis/state/room2	Publish and subscribe	Retained, QoS 1	{"occupied":true,"lights":true,"ts":epoch}

On every (re)connection the node subscribes to the two room topics and publishes its current occupancy and schedule. When a message arrives on a room topic, the JSON is parsed with cJSON and, if both fields are valid booleans, the room state is updated.

6.8 Room Automation

The room task reads both PIR sensors every 100 ms. Motion resets a hold counter and turns the room light on. When there is no motion, the counter runs for five seconds, after which the light is switched off. The room state is stored and published only when it changes. The five-second grace period prevents flicker from brief gaps in the sensor output. An administrator can also force a room's lights on or off, or return them to automatic control, with the lights command.

6.9 Serial Console

The console task reads UART0 one byte at a time, echoes characters, supports backspace, and executes the command when Enter is pressed. A 128-byte line buffer prevents overflow. An administrator session starts with the login command and expires automatically after ADMIN_TTL_S (300 seconds). Table XXII lists the working commands.

TABLE XXII

CONSOLE COMMANDS

Command	**Access**	**Description**
help	Open	Prints the command list.
status	Open	Prints time, schedule, occupancy, admin state and both rooms.
occupancy	Open	Prints the current occupancy.
who	Open	Lists the card identifiers currently inside.
schedule show	Open	Prints the schedule window.
schedule set HH:MM HH:MM	Admin	Sets, saves and publishes a new schedule; times are validated.
login <user> <pin>	Open	Starts a five-minute admin session if the PIN is correct.
logout	Open	Ends the admin session.
reboot	Open	Restarts the device.
scan <in|out> <HEX>	Open	Simulates a card tap at the entry or exit reader.
user list	Open	Lists the authorised cards.
user add <HEX> | user remove <HEX>	Admin	Authorises or removes a card; the list is saved in NVS.
lights <room1|room2> <on|off|auto>	Admin	Forces a room's lights on or off, or returns them to automatic control.

6.10 LCD Driver

The 16x2 LCD uses an HD44780 controller connected through a PCF8574 I2C expander in 4-bit mode. Every byte is sent as two nibbles. Each nibble is placed in the upper four bits of an expander byte together with the register-select, enable and backlight bits, and the enable line is pulsed high then low to latch the data. The initialisation follows the datasheet reset sequence of three function-set nibbles followed by the switch to 4-bit mode, then sets two

lines with a 5x8 font, clears the display, selects left-to-right entry and turns the display on. The idle screen shows “CoCIS Security” and the occupancy. The result screen shows ACCESS GRANTED or ACCESS DENIED and the direction and tag.

6.11 Time Synchronisation

Schedule decisions and log timestamps depend on the real time, which the firmware obtains with SNTP from pool.ntp.org and presents in the time zone EAT-3. Until the first synchronisation completes, the clock reports a time near the Unix epoch.

7. DATABASE DESIGN

The central database is the system's long-term memory. It answers the questions management cares about: who is in the building now, who came in yesterday and when they left, whether the lecture-room lights were left on, and who changed the schedule. The ESP32 keeps only what it needs to work on its own (NVS flash and the SD-card log); the database on the server is the official record.

7.1 Choice of Database Engine

SQLite 3 was chosen for the central database. It stores the whole database in one file (data/cocis.db), needs no separate database server, is built into Python, and fully supports transactions, foreign keys, unique and partial indexes, generated columns and views, all of which the design relies on. Write-Ahead Logging (WAL) mode is enabled so that read requests, such as the dashboard refreshing every three seconds, never wait for a write. Because the API uses only standard SQL, the same schema can move to PostgreSQL or MySQL if CoCIS rolls the system out to the whole college.

7.2 Entity-Relationship Diagram

Fig. 6 shows the eleven tables and how they are related. People and their movements are held in users, access_events and building_visits; rooms and lighting in rooms and room_status_logs; staff accounts in admins and admin_sessions; configuration in access_schedule and system_settings; and device health and the audit trail in devices and audit_log.

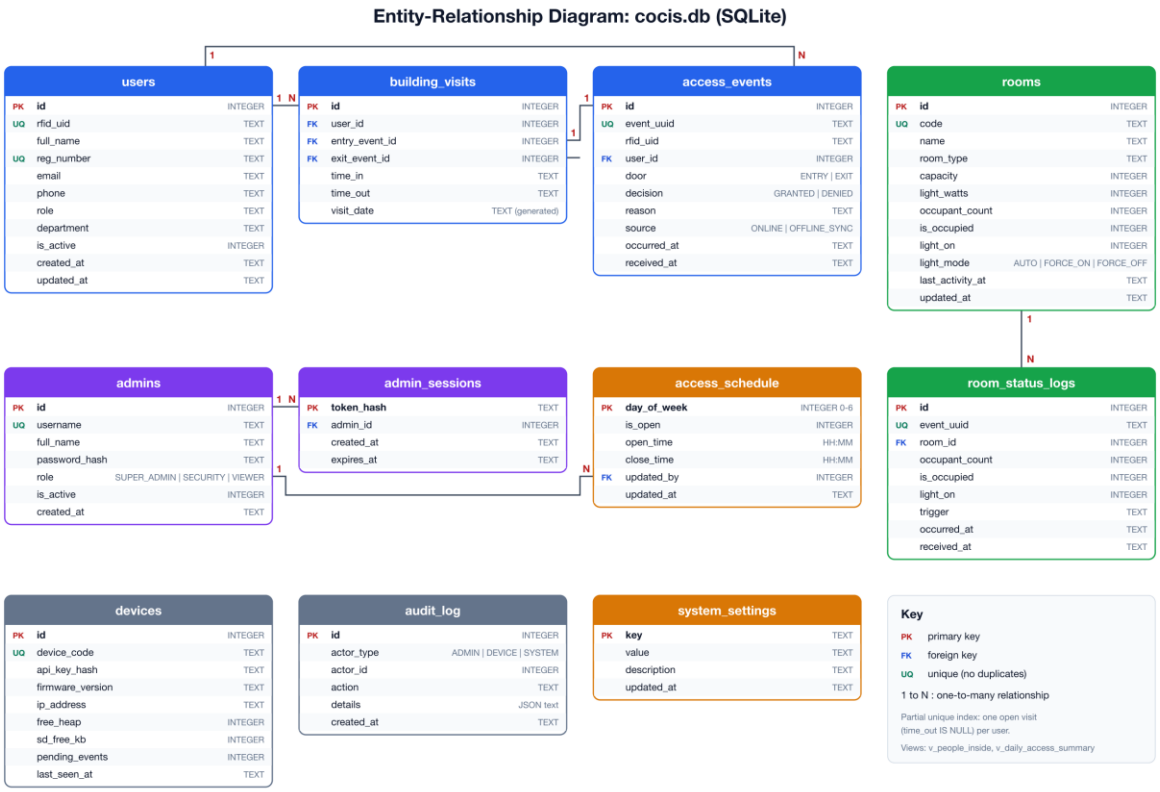


Fig. 6. Entity-relationship diagram of the cocis.db database.

- **users to access_events (one to many):** one person can scan many times. The user_id is empty for unknown tags, but the raw rfid_uid is always stored, so intruder attempts are still logged.
- **users to building_visits (one to many):** one row per stay in the building; a visit with no time_out means the person is inside now.
- **access_events to building_visits:** each visit points at the event that opened it (entry_event_id) and the event that closed it (exit_event_id).
- **rooms to room_status_logs (one to many):** every change of a room's occupancy or light state is logged, which gives the lighting energy report.
- **admins to admin_sessions and access_schedule:** who is logged in, and who last changed each day's opening hours.

7.3 Tables

Table XXIII describes each table. All times are stored as ISO 8601 text in East Africa Time (for example 2026-10-30T08:15:02), which sorts correctly and is easy to produce on the ESP32.

TABLE XXIII

TABLES OF THE CENTRAL DATABASE

Table	Holds	Key columns
users	Every person issued an RFID card	rfid_uid (unique), full_name, reg_number, role, is_active
rooms	The entrance and the two lecture rooms, with each room's live state	code (MAIN, LR1, LR2), occupant_count, light_on, light_mode, light_watts
access_events	Every RFID scan, granted or denied	event_uuid (unique), rfid_uid, user_id, door, decision, reason, source, occurred_at
building_visits	One row per stay in the building	user_id, entry_event_id, exit_event_id, time_in, time_out, visit_date
room_status_logs	History of occupancy and lighting changes	event_uuid, room_id, occupant_count, light_on, trigger, occurred_at
access_schedule	Opening hours for each day of the week	day_of_week, is_open, open_time, close_time, updated_by
system_settings	Key and value settings	emergency_mode, lockdown_mode, light_off_delay_s, config_version
admins	Staff accounts	username, password_hash (PBKDF2-SHA256), role, is_active
admin_sessions	Login tokens currently issued	token_hash (SHA-256 of the token), admin_id, expires_at
devices	Controllers allowed to send data, with their health	device_code, api_key_hash, firmware_version, last_seen_at
audit_log	Every configuration change and login attempt	actor_type, actor_id, action, details (JSON), created_at

7.4 Integrity Rules, Indexes and Views

The database itself enforces the rules, so a bug in a client cannot store inconsistent data (Table XXIV).

TABLE XXIV

INTEGRITY CONSTRAINTS, INDEXES AND VIEWS

Object	Type	Why it exists
Foreign keys (PRAGMA foreign_keys = ON)	Constraint	A visit cannot point at a user or event that does not exist.
CHECK constraints on roles, doors, decisions, reasons, light modes, triggers, counts and 0/1 flags	Constraint	Invalid values are rejected by the database, not only by the API.

Object	Type	Why it exists
CHECK (open_time < close_time)	Constraint	A schedule cannot close before it opens.
uq_one_open_visit on building_visits(user_id) WHERE time_out IS NULL	Partial unique index	A person can be inside only once: the final safeguard against race conditions.
event_uuid UNIQUE on access_events and room_status_logs	Unique index	An event delivered twice is stored once.
Indexes on event time, user, visit date, room logs and audit time	Index	Fast log searches and daily reports.
v_people_inside	View	Everyone with an open visit: answers who is inside now.
v_daily_access_summary	View	Entries, exits and refusals per day.

7.5 Initial Data

When the API starts for the first time it creates the database from database/schema.sql and loads database/seed.sql: the three rooms, a default schedule (Monday to Friday 07:00 to 22:00, Saturday 08:00 to 17:00, Sunday closed), the default settings, demonstration card holders whose UIDs match the Wokwi simulation, one administrator account and the device keys. These demonstration credentials (Table XXVIII) must be changed before any real deployment.

8. REST API AND HOW TO ACCESS IT

The REST API is the only way into the database. The MQTT bridge uses it to store the controller's events, and staff use it, through the dashboard, Swagger UI or any HTTP client, to monitor the building and change settings. It is written in Python with FastAPI, validates every request with Pydantic models, and is served by Uvicorn on port 8000.

8.1 General Conventions

TABLE XXV

GENERAL API CONVENTIONS

Item	Value
Base URL	http://127.0.0.1:8000/api/v1 on the server itself (http://<server-ip>:8000/api/v1 from the college network)
Format	JSON request and response bodies (Content-Type: application/json)
Time format	ISO 8601 local time (EAT), for example 2026-10-30T08:15:02
Device authentication	Header X-Device-Key: <device key>
Staff authentication	Header Authorization: Bearer <token>, using the token returned by POST /auth/login
Documentation	Swagger UI at http://127.0.0.1:8000/docs, ReDoc at /redoc, and the machine-readable specification at /openapi.json
Dashboard	http://127.0.0.1:8000/dashboard/

8.2 Security Model

There are two kinds of client, each with its own credential. Devices (the MQTT bridge on behalf of the entrance ESP32, or an HTTP controller) send a secret device key on every request; the server stores only its SHA-256 hash. Staff log in with a username and password, which is stored as a salted PBKDF2 hash, and receive a random bearer token that is valid for eight hours; only a hash of the token is stored. Every management endpoint checks the token and the role (Table XXIX), and all configuration changes, logins and failed logins are written to the audit log.

8.3 Endpoint Summary

Table XXVI lists every endpoint. The paths are relative to the base URL.

TABLE XXVI

SUMMARY OF API ENDPOINTS

Method	**Path**	**Access**	**Purpose**
GET	/health	None	Check that the API is running
POST	/device/relay	Device key	Store events decided by the MQTT controller, relayed by the bridge
POST	/device/heartbeat	Device key	Report controller health; learn config_version and lockdown_mode
POST	/device/access/check	Device key	Decide and record a scan (HTTP controller)
POST	/device/rooms/status	Device key	Report a room change (HTTP controller)
POST	/device/events/batch	Device key	Upload events stored while offline (HTTP controller)
GET	/device/config	Device key	Download schedule, cards and settings (HTTP controller)
POST	/auth/login	None	Log in and receive a bearer token
POST	/auth/token	None	The same login as a form, used by Swagger UI's Authorize button
POST	/auth/logout	Any role	Invalidate the token
GET	/auth/me	Any role	Return the logged-in account
GET	/users, /users/{id}	VIEWER	List, search or read card holders
POST	/users	SUPER_ADMIN	Register a new card holder
PATCH	/users/{id}	SUPER_ADMIN	Change details, re-issue a card, activate or deactivate
DELETE	/users/{id}	SUPER_ADMIN	Deactivate a card holder (history is kept)
GET	/occupancy	VIEWER	People inside now and the state of each lecture room
GET	/access-events	VIEWER	Search the access log with filters and paging
GET	/visits	VIEWER	Time in and time out per person per day
GET	/rooms, /rooms/{code}/history	VIEWER	Rooms with their live state, and a room's changes on a given day
PUT	/rooms/{code}/light-mode	SECURITY	AUTO, FORCE_ON or FORCE_OFF light override
GET	/schedule	VIEWER	Weekly opening hours
PUT	/schedule	SUPER_ADMIN	Change one or more days of the schedule

Method	Path	Access	Purpose
GET	/settings	VIEWER	All system settings
PATCH	/settings	SUPER_ADMIN	Change the light-off delay or heartbeat interval
PUT	/settings/emergency	SECURITY	Emergency mode (all doors open) for the HTTP controller
PUT	/settings/lockdown	SECURITY	Lockdown on or off; forwarded to the ESP32 by the bridge
GET	/reports/daily	VIEWER	Daily totals, peak occupancy and lighting energy
GET	/devices	VIEWER	Controller health and online status
GET	/audit-log	SUPER_ADMIN	Recent configuration changes and logins
GET	/db/tables, /db/{table}	SUPER_ADMIN	Read-only view of the database tables (secrets hidden)

The API answers with standard HTTP status codes: 200 for success, 201 when a user is created, 401 for a missing or wrong device key, password or token, 403 when the role is not allowed, 404 when a user or room does not exist, 409 when a card or registration number is already registered, and 422 when validation fails. Errors always have the same shape, for example {"detail": "Requires role SUPER_ADMIN or higher"}.

8.4 Starting the API

The backend needs only Python 3.10 or newer. Ready-made scripts in the backend folder do the setup and start-up (Table XXVII). On first start the database file data/cocis.db is created automatically and the browser opens Swagger UI.

TABLE XXVII

SCRIPTS FOR INSTALLING, STARTING AND TESTING THE BACKEND

Task	Windows	macOS or Linux
Install the Python packages (once)	setup.bat	./setup.sh
Start the API, database and dashboard	start.bat	./start.sh
Connect the entrance ESP32 through MQTT	bridge.bat	./bridge.sh
Send the entrance ESP32's messages without Wokwi	entrance_sim.bat	./entrance_sim.sh
Run the automated tests	test.bat	./test.sh
Start again with a fresh database	reset_db.bat	Delete data/cocis.db

8.5 Accessing the API with Swagger UI

Swagger UI is the easiest way to explore and test the API: it lists every endpoint with its request and response format and lets a tester call it from the browser (Fig. 7).

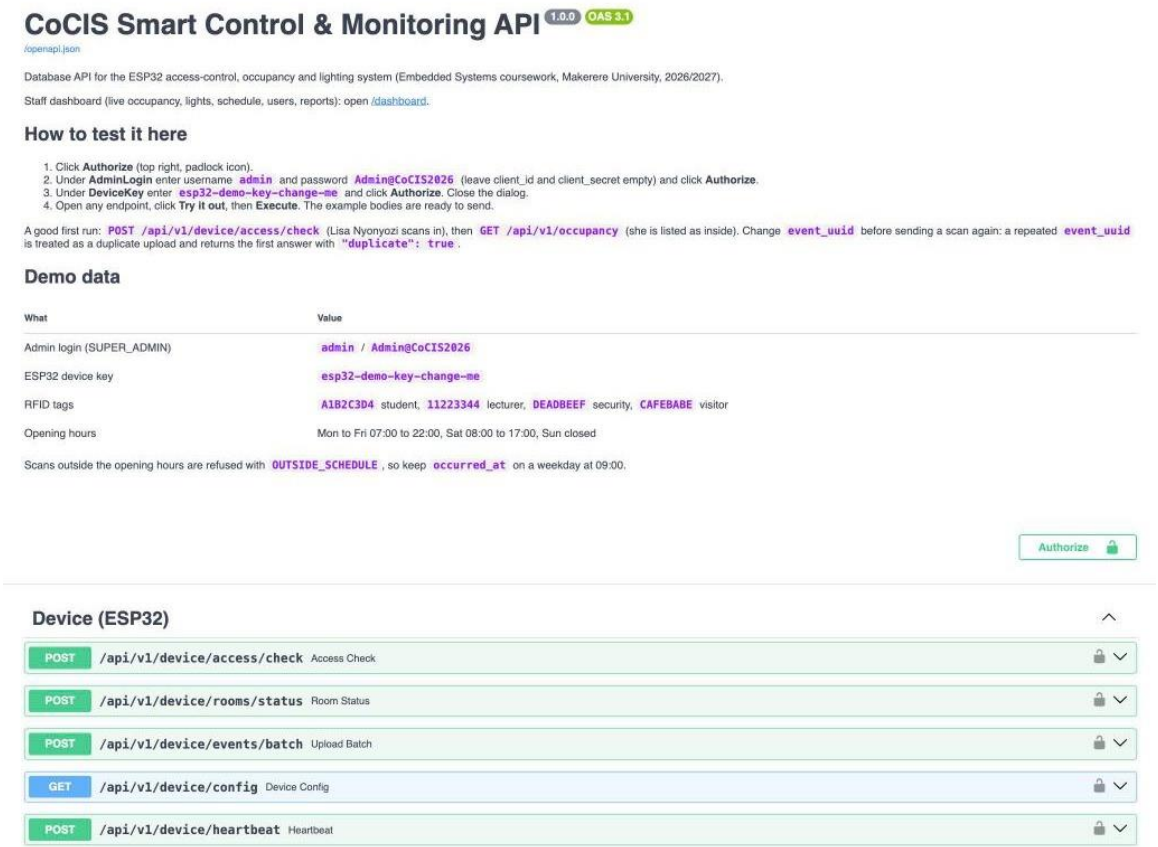


Fig. 7. Swagger UI at /docs: instructions, demonstration data and the device endpoints.

1. Start the API with start.bat and open http://127.0.0.1:8000/docs.
2. Click Authorize. Under AdminLogin enter the administrator username and password and click Authorize; under DeviceKey enter the device key and click Authorize (Fig. 8). Close the dialog.
3. Open an endpoint, for example POST /device/access/check, click Try it out, edit the example body if needed and click Execute.
4. Read the response below the request (Fig. 9). Swagger UI also shows the equivalent curl command, which can be copied into a terminal.

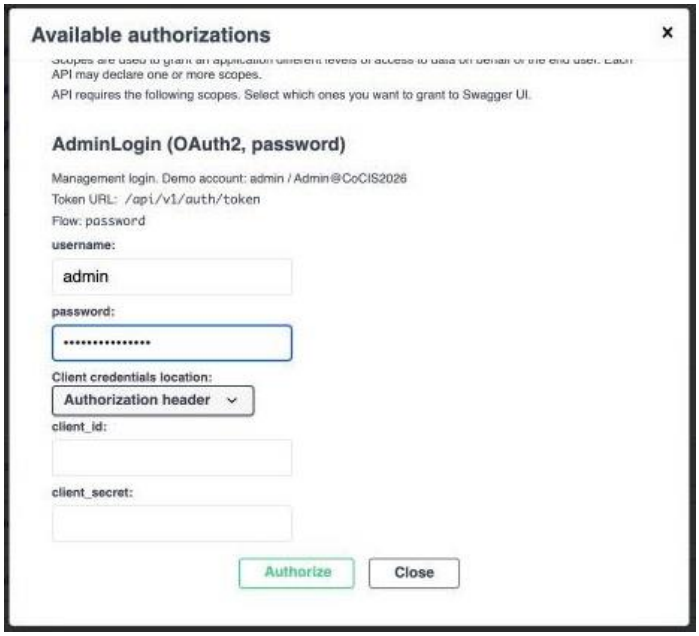


Fig. 8. Swagger UI Authorize dialog: device key and administrator login.

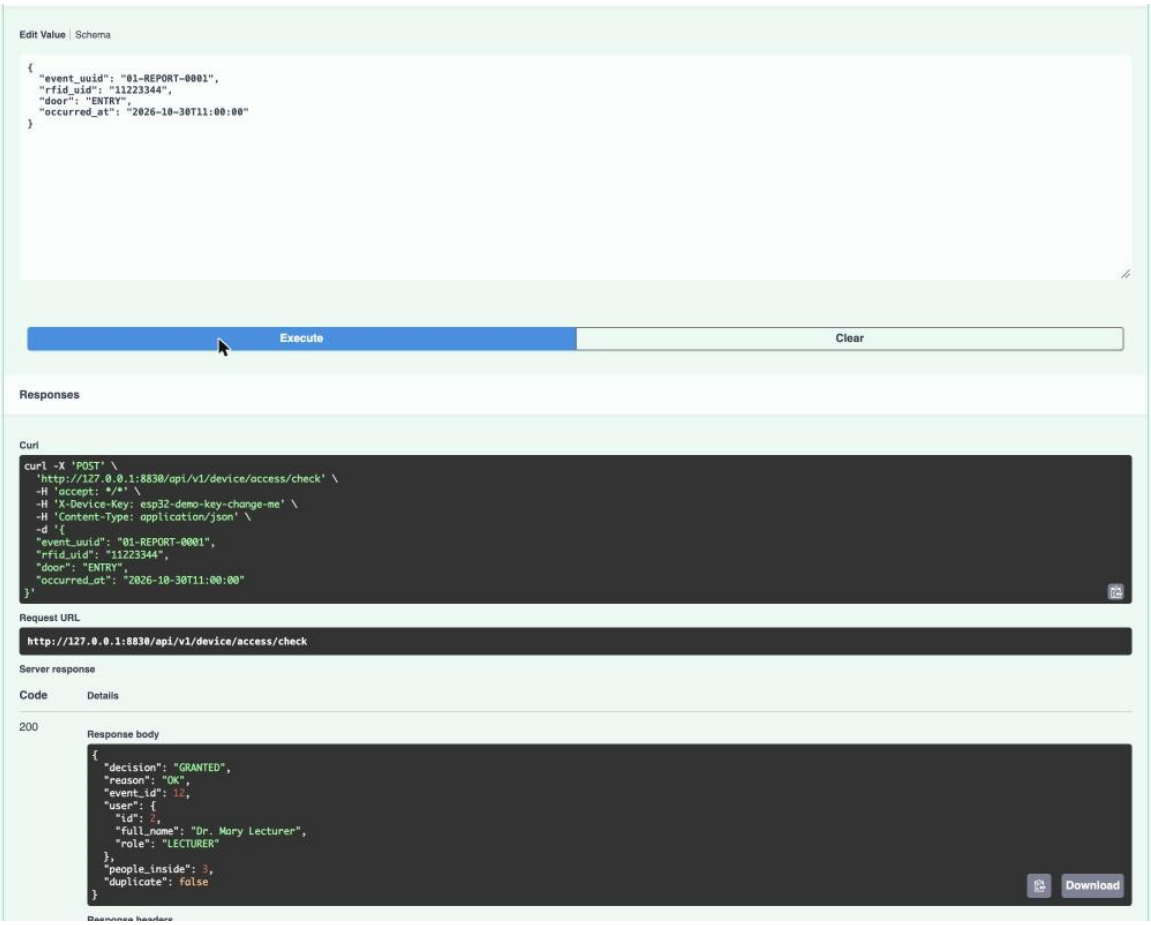


Fig. 9. Executing POST /device/access/check in Swagger UI: the request, the curl command and the GRANTED response.

8.6 Accessing the API from Other Clients

Any HTTP client (curl, Postman, a Python script or a future mobile app) can use the API in the same way. A staff client first logs in to obtain a token, then sends the token with every request:

```
curl -X POST http://127.0.0.1:8000/api/v1/auth/login \
    -H "Content-Type: application/json" \
    -d '{"username": "admin", "password": "Admin@CoCIS2026"}'

{"access_token": "q5k...Zt0", "token_type": "bearer",
 "expires_at": "2026-10-30T16:15:00", "role": "SUPER_ADMIN", ...}

curl http://127.0.0.1:8000/api/v1/occupancy \
    -H "Authorization: Bearer q5k...Zt0"
```

The occupancy response lists the people inside and the state of each lecture room:

```
{ "people_inside_count": 2,
  "people_inside": [ { "full_name": "Lisa Nyonyozi", "role": "STUDENT",
                       "rfid_uid": "A1B2C3D4", "time_in": "2026-10-30T08:15:02" } ],
  "rooms": [ { "code": "LR1", "occupant_count": 1, "is_occupied": 1,
               "light_on": 1, "light_mode": "AUTO" } ],
  "emergency_mode": false, "lockdown_mode": false }
```

A device client sends its key in the X-Device-Key header instead. This is the request the MQTT bridge makes for every scan from the entrance ESP32; the API answers with the identifiers it stored, and lists any event it already had under duplicates:

```
POST /api/v1/device/relay
X-Device-Key: mqtt-bridge-key-change-me

{ "access_events": [ { "event_uuid": "MQ-11223344-IN-1793636493",
     "rfid_uid": "11223344", "door": "ENTRY", "decision": "GRANTED",
     "reason": "OK", "occurred_at": "2026-11-02T19:21:33" } ],
  "room_events": [] }

{ "accepted": ["MQ-11223344-IN-1793636493"], "duplicates": [] }
```

8.7 Demonstration Credentials

TABLE XXVIII

DEMONSTRATION CREDENTIALS (CHANGE BEFORE A REAL DEPLOYMENT)

Credential	**Value**	**Used by**
Administrator	admin / Admin@CoCIS2026 (SUPER_ADMIN)	Dashboard, Swagger UI, other HTTP clients
Device key (bridge)	mqtt-bridge-key-change-me (device ESP32-MQTT-ENTRANCE)	MQTT bridge
Device key (HTTP)	esp32-demo-key-change-me (device ESP32-CTRL-01)	HTTP reference controller, virtual ESP32, Swagger UI

Credential	Value	Used by
ESP32 admin PIN	login admin 1234	Entrance firmware serial console
Demonstration cards	11223344 and 01020304 (authorised on the ESP32); A1B2C3D4, DEADBEEF and CAFEBABE in the database	Wokwi simulation, dashboard

9. MANAGEMENT DASHBOARD

The management dashboard is the web application through which college management and security staff monitor and manage the system without using the serial console. It is written in plain HTML, CSS and JavaScript (dashboard/index.html, app.js and style.css), is served by the REST API at /dashboard/, and needs no installation: staff open it in any web browser, including on a phone. The dashboard calls only the REST API, with the same login and role checks as Swagger UI, and refreshes its live views every three seconds, so a card scanned at the ESP32 appears on screen within about three seconds.

9.1 Accessing the Dashboard and Logging In

With the API running (start.bat), staff open http://127.0.0.1:8000/dashboard/ on the server, or http://<server-ip>:8000/dashboard/ from another computer on the college network, and log in with a username and password (Fig. 10). The API checks the password hash and issues a token that is valid for eight hours, and every login is recorded in the audit log.



Fig. 10. Dashboard login page.

9.2 Roles and Permissions

The role on each staff account decides what that person may see and change (Table XXIX). The same roles protect the REST API, so a user cannot bypass them by calling the API directly.

TABLE XXIX

DASHBOARD AND API ROLES

Role	Can view	Can change
VIEWER	Occupancy, lecture rooms, access events, visits, schedule, reports and controller status	Nothing
SECURITY	Everything a VIEWER can see	Emergency mode, lockdown and room light modes
SUPER_ADMIN	Everything, including the audit log and the Database tab	Everything above, plus card holders, the schedule and system settings

9.3 Overview Tab

The Overview tab is the live view for security staff (Fig. 11). It shows the number of people inside, the emergency switch, a card for each lecture room with its occupancy, light state and light mode buttons (AUTO, FORCE_ON and FORCE_OFF), the controller's status (online, last seen, pending events and firmware), the people inside the building, and the latest access events with their decision and reason. Denied scans are highlighted, so an unknown card stands out at once.

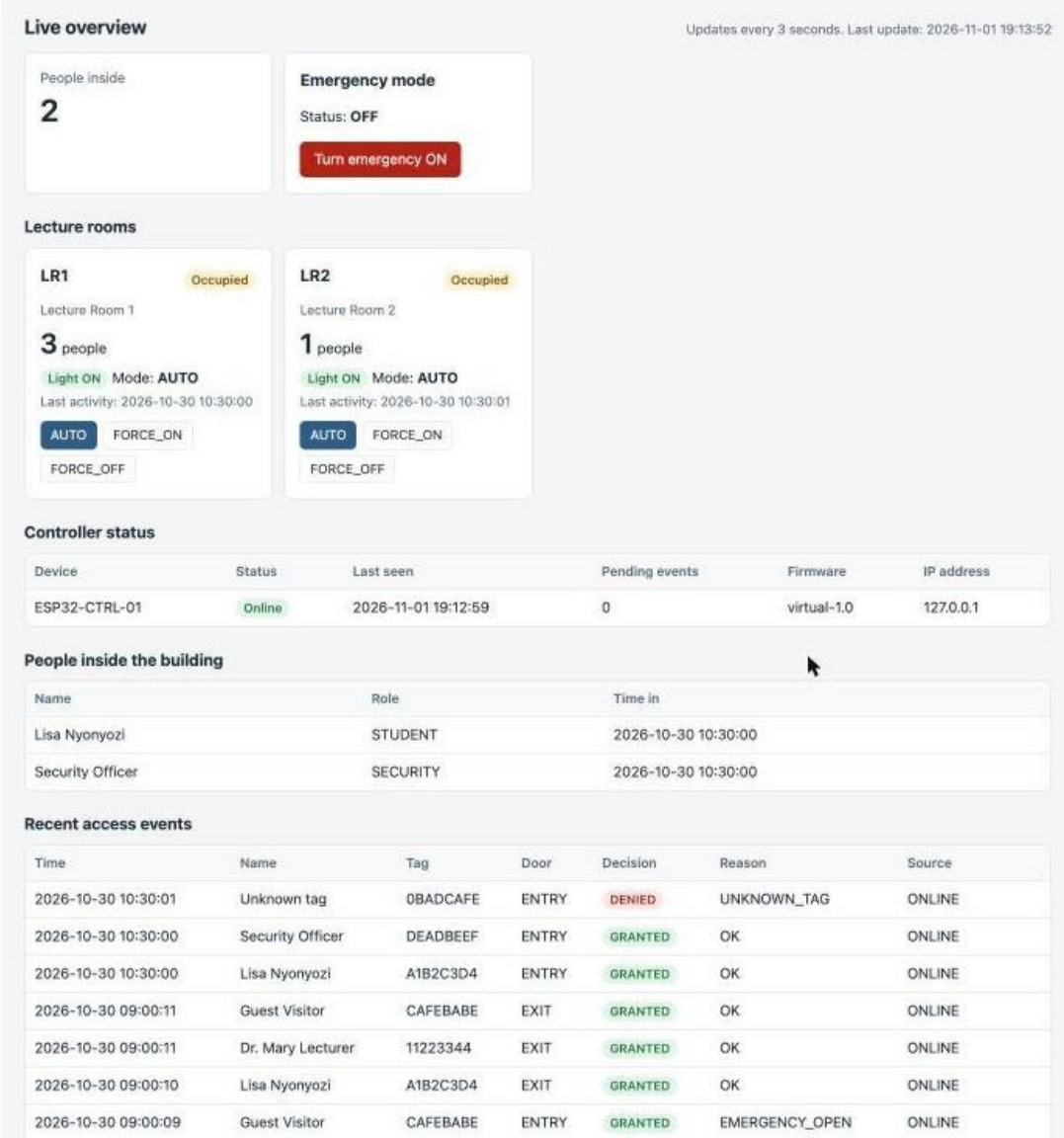


Fig. 11. Dashboard Overview tab: occupancy, lecture rooms, controller status and recent access events.

9.4 Schedule Tab

The Schedule tab sets the opening hours for each day of the week (Fig. 12). Only a SUPER_ADMIN can save, a closing time before the opening time is refused with a clear message, and every change is audited.

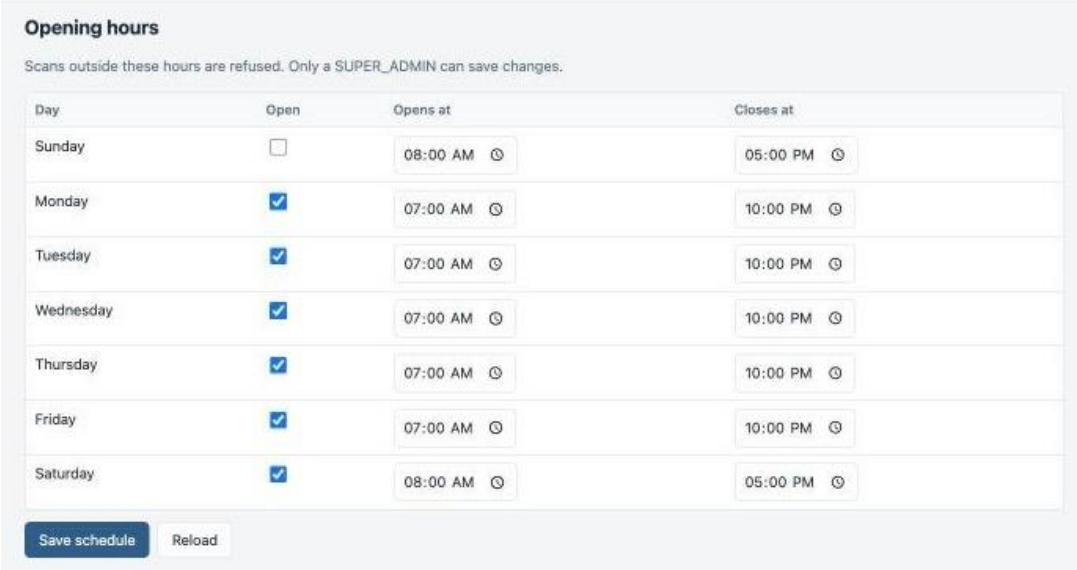


Fig. 12. Dashboard Schedule tab: opening hours per day.

9.5 Users Tab

The Users tab registers new card holders and deactivates lost or expired cards (Fig. 13). A deactivated card is refused at once, and its history is kept for reports.

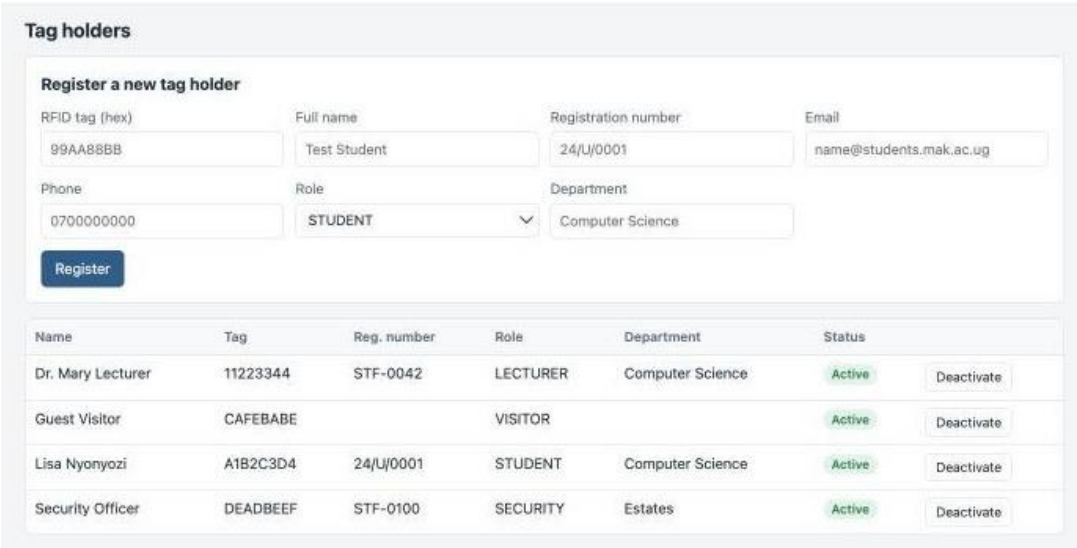


Fig. 13. Dashboard Users tab: registering and deactivating card holders.

9.6 Reports Tab

The Reports tab gives the daily totals of entries, exits and refusals, refusals by reason, the number of unique visitors, the busiest moment of the day, and the hours and energy (kWh) used by each room's lights, with a bar chart (Fig. 14). The lighting figures support the project's aim of reducing the college's electricity costs.

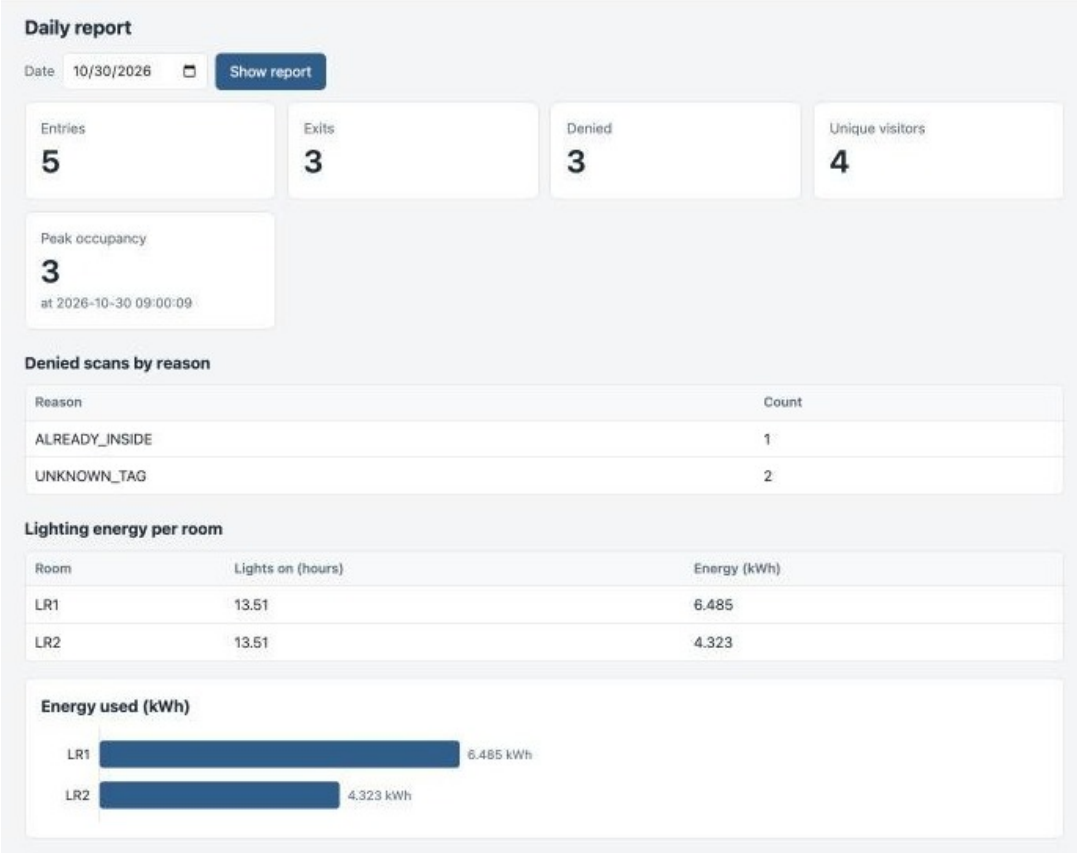


Fig. 14. Dashboard Reports tab: daily totals, peak occupancy and lighting energy.

9.7 Audit Log Tab

The Audit log tab lists every login and change with the staff member who made it and the reason given (Fig. 15), so every configuration change can be traced.

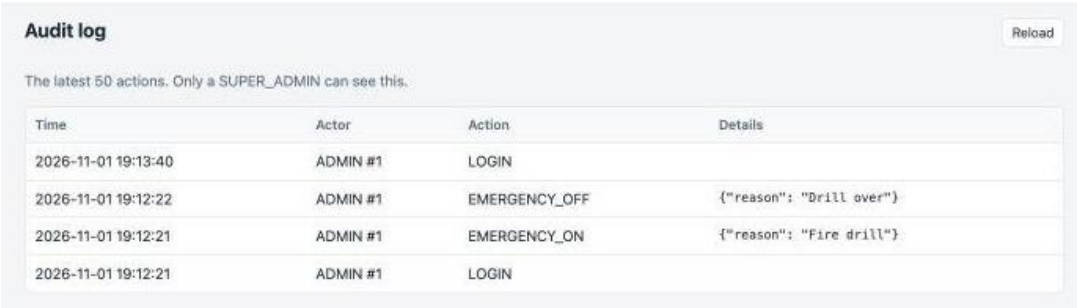
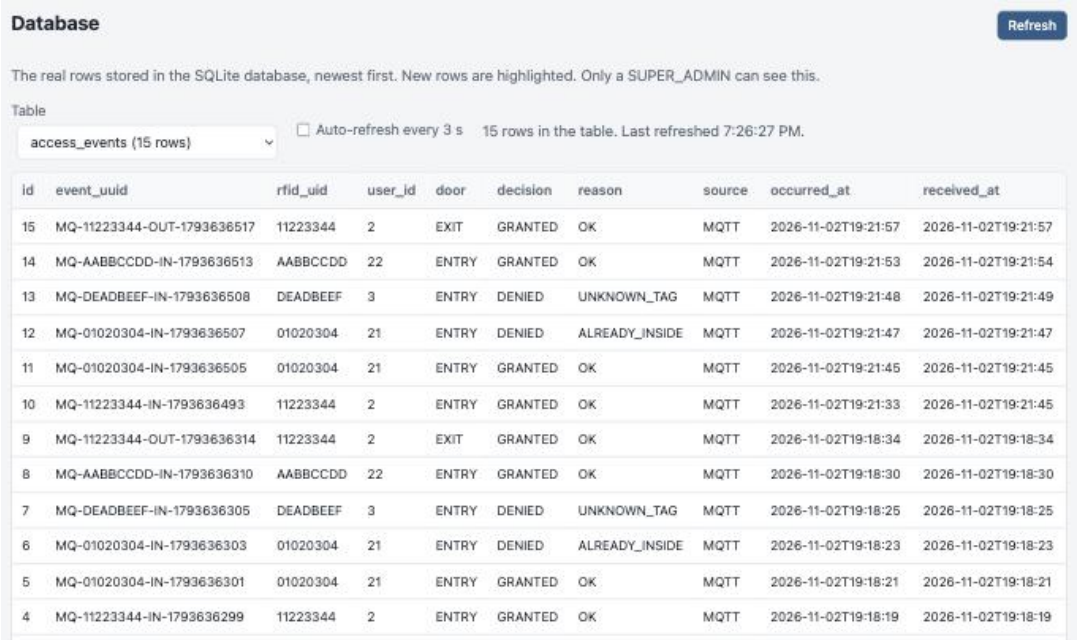


Fig. 15. Dashboard Audit log tab.

9.8 Database Tab

The Database tab shows the real rows of any table, newest first, with a Refresh button and an optional three-second auto-refresh; new rows are highlighted. It proves that every scan and room change from the ESP32 reaches the database: Fig. 16 shows access events stored from

the entrance controller over MQTT, and Fig. 17 shows the lecture-room motion changes. Only a SUPER_ADMIN can open this tab, and password and key hashes are never shown.



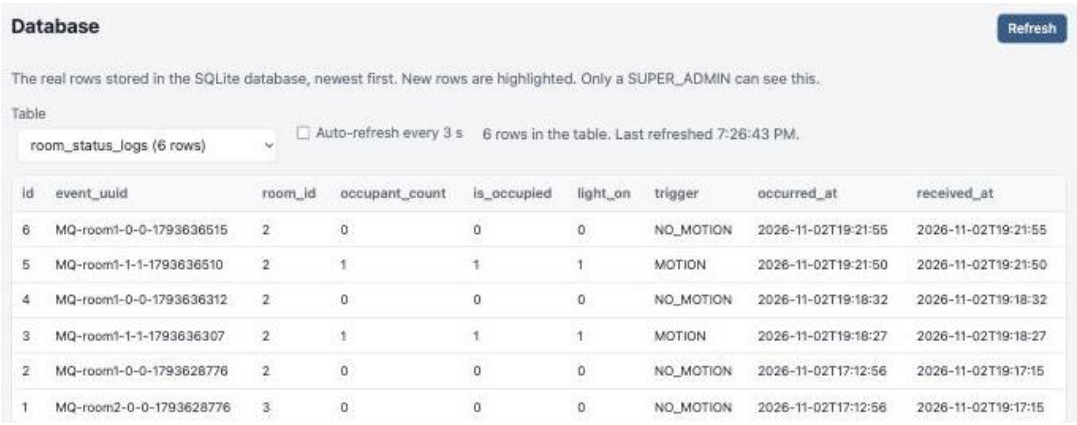
Database Refresh

The real rows stored in the SQLite database, newest first. New rows are highlighted. Only a SUPER_ADMIN can see this.

Table: access_events (15 rows) — Auto-refresh every 3 s — 15 rows in the table. Last refreshed 7:26:27 PM.

id	event_uuid	rfid_uid	user_id	door	decision	reason	source	occurred_at	received_at
15	MQ-11223344-OUT-1793636517	11223344	2	EXIT	GRANTED	OK	MQTT	2026-11-02T19:21:57	2026-11-02T19:21:57
14	MQ-AABBCCDD-IN-1793636513	AABBCCDD	22	ENTRY	GRANTED	OK	MQTT	2026-11-02T19:21:53	2026-11-02T19:21:54
13	MQ-DEADBEEF-IN-1793636508	DEADBEEF	3	ENTRY	DENIED	UNKNOWN_TAG	MQTT	2026-11-02T19:21:48	2026-11-02T19:21:49
12	MQ-01020304-IN-1793636507	01020304	21	ENTRY	DENIED	ALREADY_INSIDE	MQTT	2026-11-02T19:21:47	2026-11-02T19:21:47
11	MQ-01020304-IN-1793636505	01020304	21	ENTRY	GRANTED	OK	MQTT	2026-11-02T19:21:45	2026-11-02T19:21:45
10	MQ-11223344-IN-1793636493	11223344	2	ENTRY	GRANTED	OK	MQTT	2026-11-02T19:21:33	2026-11-02T19:21:45
9	MQ-11223344-OUT-1793636314	11223344	2	EXIT	GRANTED	OK	MQTT	2026-11-02T19:18:34	2026-11-02T19:18:34
8	MQ-AABBCCDD-IN-1793636310	AABBCCDD	22	ENTRY	GRANTED	OK	MQTT	2026-11-02T19:18:30	2026-11-02T19:18:30
7	MQ-DEADBEEF-IN-1793636305	DEADBEEF	3	ENTRY	DENIED	UNKNOWN_TAG	MQTT	2026-11-02T19:18:25	2026-11-02T19:18:25
6	MQ-01020304-IN-1793636303	01020304	21	ENTRY	DENIED	ALREADY_INSIDE	MQTT	2026-11-02T19:18:23	2026-11-02T19:18:23
5	MQ-01020304-IN-1793636301	01020304	21	ENTRY	GRANTED	OK	MQTT	2026-11-02T19:18:21	2026-11-02T19:18:21
4	MQ-11223344-IN-1793636299	11223344	2	ENTRY	GRANTED	OK	MQTT	2026-11-02T19:18:19	2026-11-02T19:18:19

Fig. 16. Database tab: access events stored from the entrance ESP32 over MQTT.



Database Refresh

The real rows stored in the SQLite database, newest first. New rows are highlighted. Only a SUPER_ADMIN can see this.

Table: room_status_logs (6 rows) — Auto-refresh every 3 s — 6 rows in the table. Last refreshed 7:26:43 PM.

id	event_uuid	room_id	occupant_count	is_occupied	light_on	trigger	occurred_at	received_at
6	MQ-room1-0-0-1793636515	2	0	0	0	NO_MOTION	2026-11-02T19:21:55	2026-11-02T19:21:55
5	MQ-room1-1-1-1793636510	2	1	1	1	MOTION	2026-11-02T19:21:50	2026-11-02T19:21:50
4	MQ-room1-0-0-1793636312	2	0	0	0	NO_MOTION	2026-11-02T19:18:32	2026-11-02T19:18:32
3	MQ-room1-1-1-1793636307	2	1	1	1	MOTION	2026-11-02T19:18:27	2026-11-02T19:18:27
2	MQ-room1-0-0-1793628776	2	0	0	0	NO_MOTION	2026-11-02T17:12:56	2026-11-02T19:17:15
1	MQ-room2-0-0-1793628776	3	0	0	0	NO_MOTION	2026-11-02T17:12:56	2026-11-02T19:17:15

Fig. 17. Database tab: room status changes (MOTION and NO_MOTION) stored from the entrance ESP32.

10. IMPLEMENTED FEATURES

- Dual-reader RFID access control with an authorised card list.
- Configurable opening-hours schedule with support for windows that cross midnight.
- Anti-passback enforcement (no double entry, no exit without entry).
- Always-permitted exit, with an OK_AFTER_HOURS result for exits outside the schedule.
- Servo-driven entry and exit doors operated through a queue.
- Multi-channel feedback: green and red LEDs, buzzer patterns, and LCD messages.
- Live occupancy tracking and an inside-list of card identifiers.
- Persistent storage of schedule and inside list in NVS.
- CSV audit log on an SD card with graceful fallback if the card is absent.
- MQTT publishing of occupancy, events, schedule and room status as JSON, with retained state messages.
- MQTT subscription for room-state messages.
- Motion-controlled lighting for two rooms with a five-second grace period.
- Serial command console with PIN-protected administrative actions and automatic session expiry.
- Network time synchronisation with an East Africa Time zone setting.
- Mutex-protected shared state and a queue-based door task for safe concurrency.
- Web management dashboard with live occupancy, lecture room status, opening hours, card holders, daily reports, an audit log and a live database view.
- MQTT bridge, REST API and SQLite database that store every scan and room change exactly once.
- Role-based staff access (VIEWER, SECURITY, SUPER_ADMIN) with token login and an audit trail of every change.
- Card management (user add and user remove) and manual light overrides from the serial console.

11. SYSTEM OPERATION

11.1 Normal Operation

After power-up the LCD shows the welcome screen with the occupancy. The system waits for cards. When an authorised card is presented to the entry reader within the opening hours, the green LED lights, the buzzer gives a short beep, the entry servo opens the door for three seconds, the LCD shows ACCESS GRANTED with the direction and tag, and the occupancy increases. Presenting the same card at the exit reader opens the exit door, removes the card from the inside list and decreases the occupancy. An unknown card, a second entry, an exit without entry, or an entry outside the schedule produces the red LED, a half-second buzzer tone, no door movement and an ACCESS DENIED message. Every attempt is written to the SD log, printed on the console and published to MQTT.

11.2 Administrator Operation

The administrator opens the serial monitor and types help. After logging in with the login command, the schedule can be changed with schedule set, for example schedule set 07:00 21:00. The new schedule is stored in flash and published to MQTT. The status command shows the full system state at any time.

11.3 Monitoring

A remote client subscribes to the cocis topics on the same broker to display occupancy, a live feed of access events, the active schedule and each room’s occupied and light state. Because state topics are retained, the client sees current values immediately on connecting.

In this project that client is the MQTT bridge, which stores the messages in the database so that the management dashboard (Section 9) can display them.

11.4 Dashboard Operation

To run the backend and the dashboard on a Windows laptop:

1. In the backend folder, run setup.bat once to install the Python packages.
2. Run start.bat to start the REST API, the database and the dashboard, and keep the window open.
3. Run bridge.bat to connect the entrance controller's MQTT messages to the database.

4. Start the firmware in Wokwi (or run entrance_sim.bat, which sends the same messages).
5. Open http://127.0.0.1:8000/dashboard/ in a web browser and log in with the administrator account.
6. Tap a card in Wokwi: the scan appears in the Overview and Database tabs within three seconds.

On macOS or Linux the equivalent scripts are setup.sh, start.sh and bridge.sh. The steps and the demonstration accounts are also described in the README files of the code repository.

12. TESTING AND RESULTS

The system was tested at three levels: automated tests of the database, API and bridge; an end-to-end test of the complete path over the public MQTT broker; and demonstrations with the reference HTTP controller and the virtual ESP32.

12.1 Automated Database, API and Bridge Tests

The backend has an automated test suite (tests/test_api.py, tests/test_bridge.py and tests/test_simulator.py, run with pytest through test.bat) that starts the API on a fresh, empty database and exercises it exactly as the bridge, a controller and a staff member would. All 22 tests pass (Table XXX).

TABLE XXX

AUTOMATED TEST RESULTS

#	Test	What it proves	Result
1	Health check	The API starts and creates the database.	Pass
2	Wrong device key	Device endpoints reject unknown devices (401).	Pass
3	Entry, exit and anti-passback	An entry opens a visit, a second entry is refused, an exit closes the visit and a second exit is refused.	Pass
4	Unknown card and schedule	Unknown cards are refused; entry is refused at 23:00 and on Sunday.	Pass
5	Exit outside the schedule	The API's own decision path refuses an exit after closing time.	Pass
6	Retry is idempotent	Re-sending the same event_uuid returns the original decision and stores no duplicate.	Pass
7	Concurrent entries	Eight simultaneous scans of one card give exactly one grant and one visit.	Pass
8	Offline batch upload	Out-of-order offline events are stored correctly, a re-upload gives only duplicates, and lighting energy is computed.	Pass
9	Schedule change	A change raises config_version; opening after closing is rejected (422).	Pass
10	Role check	A VIEWER cannot change the schedule (403); a wrong password is refused (401).	Pass
11	Card registration and deactivation	A duplicate card is rejected (409); a deactivated card is refused.	Pass
12	Emergency mode	All doors open, even for unknown cards after hours; switching on and off is audited.	Pass

#	Test	What it proves	Result
13	Heartbeat	Controller health is stored and the controller is shown as online.	Pass
14	Room counts	A room stays occupied with lights on while its count is above zero; negative counts are rejected (422).	Pass
15	Swagger UI login	The Authorize form returns a working token; a wrong password is refused.	Pass
16	Dashboard served	The dashboard page, script and style sheet are served by the API.	Pass
17	Database viewer	Rows are read-only, hashes are never returned and unknown tables are refused.	Pass
18	Firmware results mapped	All six result codes of state.c are stored faithfully; a 1970 timestamp is replaced.	Pass
19	PIR rooms stored	Occupied and vacant become counts 1 and 0 with the triggers MOTION and NO_MOTION.	Pass
20	Relay stores once	Events through /device/relay have source MQTT and open a visit; a second delivery is a duplicate.	Pass
21	Lockdown reaches the bridge	Lockdown appears in the heartbeat reply and the occupancy view, and can be turned off.	Pass
22	Virtual ESP32 end to end	Online and offline decisions, a queue upload and a repeated upload leave exactly the right people inside, with no duplicates.	Pass

12.2 End-to-End Test over the MQTT Broker

The complete path was tested over the internet on 2 November 2026. The entrance controller's messages were published to broker.hivemq.com, the bridge relayed them to the API (Fig. 18), and the database and the dashboard were checked (Fig. 16 and Fig. 17). Table XXXI lists the results.

TABLE XXXI

END-TO-END TEST OVER THE MQTT BROKER

#	Message published	Expected in the database	Result
M1	scan in 11223344 (result OK)	GRANTED entry, source MQTT, visit opened	Pass
M2	scan in 01020304 twice	First GRANTED, second DENIED ALREADY_INSIDE	Pass
M3	scan in DEADBEEF (not authorised on the ESP32)	DENIED UNKNOWN_TAG	Pass

#	Message published	Expected in the database	Result
M4	Room 1 motion, then vacant	room_status_logs MOTION then NO_MOTION; LR1 light on then off	Pass
M5	user add AABBCCDD, then scan in AABBCCDD	Card registered in users, then GRANTED entry	Pass
M6	scan out 11223344	GRANTED exit, visit closed	Pass
M7	Retained messages from an earlier run	Ignored, nothing stored	Pass
M8	Controller status	ESP32-MQTT-ENTRANCE shown online on the dashboard	Pass

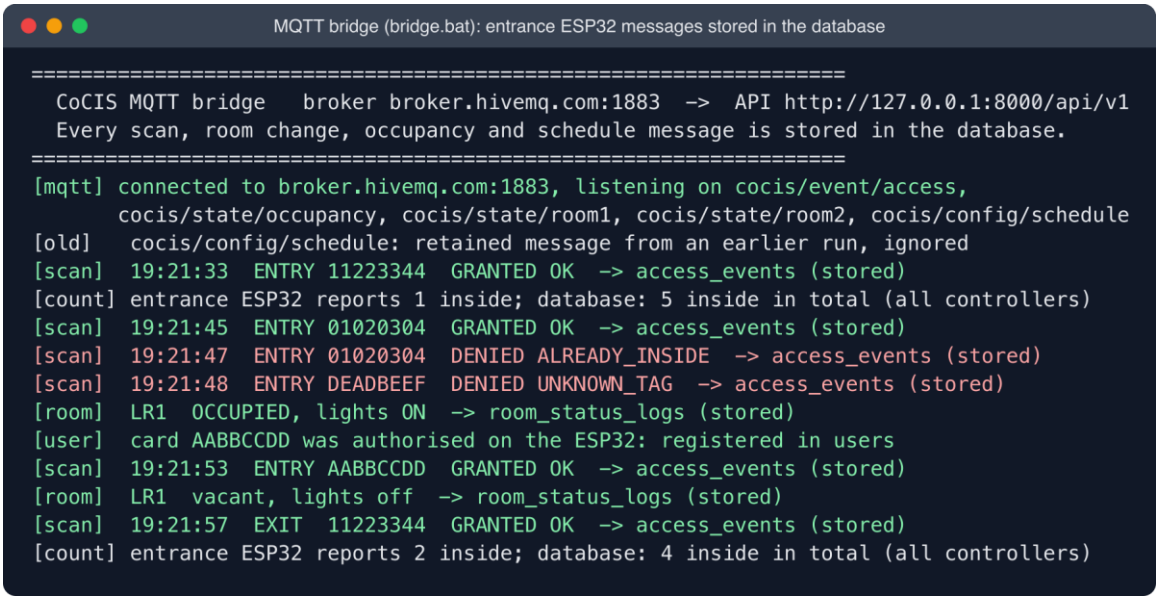


Fig. 18. MQTT bridge window: messages from the entrance ESP32 stored in the database.

12.3 Reference HTTP Controller and Virtual ESP32

Two extra tools test the server without the entrance hardware. A reference HTTP controller (backend/reference-firmware) calls the API directly, keeps working while the network is down and uploads its queue when it returns (Fig. 19). The virtual ESP32 (demo.bat) runs the same story as a scripted demonstration: online decisions by the API, anti-passback, an unknown card, people walking into a lecture room, loss of Wi-Fi, and an upload with no duplicates (Fig. 20).

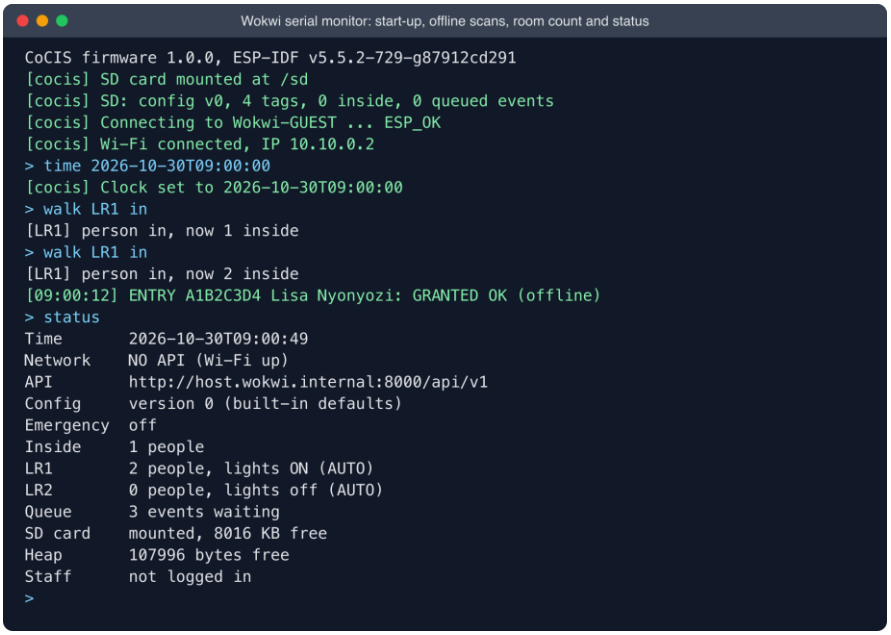


Fig. 19. Reference HTTP controller in Wokwi: start-up, offline decisions, room count and status.

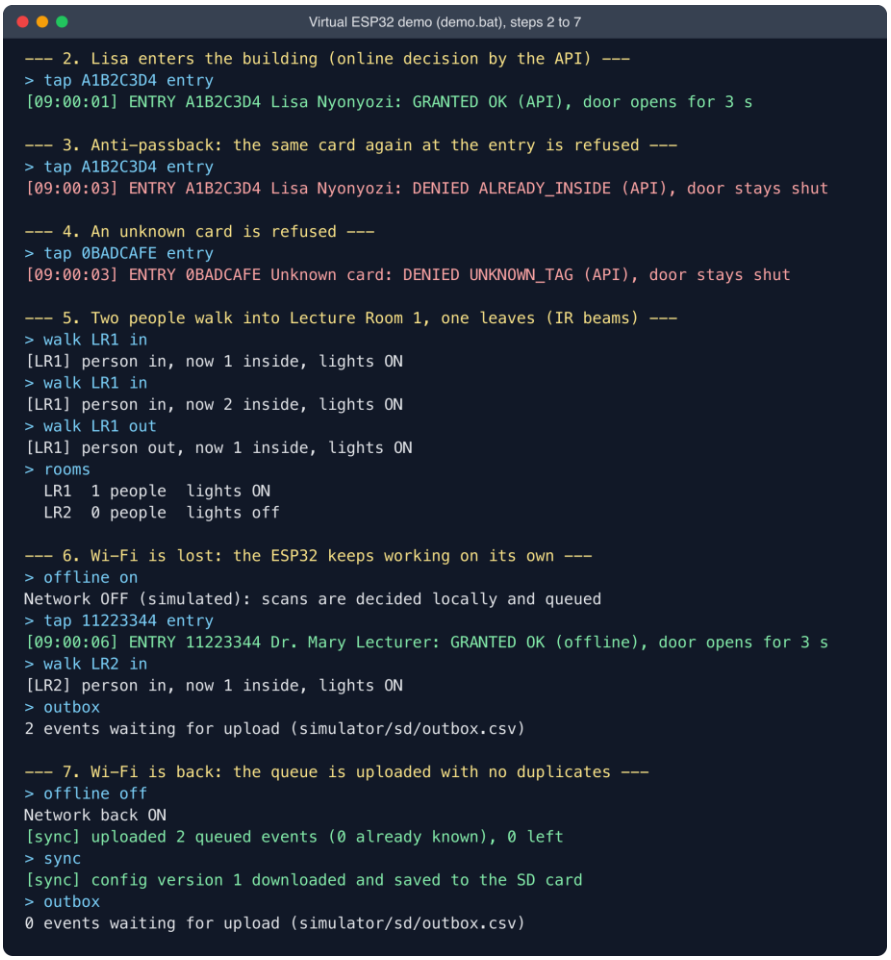


Fig. 20. Virtual ESP32 demonstration (demo.bat), steps 2 to 7.

13. LIMITATIONS AND FUTURE WORK

13.1 Known Limitations

- Initialisation order: the state module loads the saved schedule and inside list before the storage module initialises NVS, so on a normal start these values may not be restored. Calling the storage initialisation first resolves this.
- If no schedule has ever been saved, the system starts with the schedule 00:00 to 00:00, which means access is always allowed, rather than the intended 07:00 to 21:00.
- The administrator PIN is fixed in the source code, so changing it requires re-flashing.
- The Wi-Fi start-up uses a fixed three-second wait and has no reconnect handler.
- The RFID task pauses for three seconds after each scan to keep the LCD message visible, during which the other reader is not polled.
- The lockdown handler compares 19 characters of the topic, but cocis/cmd/entrance has 18, so LOCKDOWN and UNLOCK commands from the server are ignored until this value is corrected.
- The MQTT broker is public and unauthenticated, so the topics are readable and writable by anyone.
- Some commands listed by the help text (logs, sd dump, config save) are not implemented.
- The OLED driver files (oled.c, oled.h) are present but not part of the build.

13.2 Future Work

- Store the administrator PIN as a hash in NVS instead of in the source code.
- Use TLS and broker authentication for MQTT.
- Replace fixed delays with event-driven Wi-Fi handling and non-blocking display timers.
- Correct the lockdown topic check so that a lockdown set by security staff reaches the controller.
- Extend the management dashboard with push notifications and weekly occupancy trends.
- Add over-the-air firmware updates and a larger authorised card database.

14. CONCLUSION

The CoCIS Entrance Node integrates RFID authentication, schedule and anti-passback rules, door actuation, multi-channel feedback, occupancy monitoring, persistent storage, audit logging, MQTT reporting, automated motion-based room lighting and a serial administration console in a single ESP32 firmware. The modular design with a mutex-protected state module, a queue-driven door task and a central configuration header keeps the code understandable and safe under concurrency. The system is demonstrated in the Wokwi simulator, and the limitations identified in this report provide a clear path for improvement toward a production-ready deployment. The management dashboard, supported by the MQTT bridge, the REST API and the SQLite database, turns the node's messages into a permanent record that staff can monitor and manage from a web browser.

REFERENCES

[1] Espressif Systems, “ESP-IDF Programming Guide.” [Online]. Available: ESP-IDF framework documentation

[2] Espressif Systems, “ESP-MQTT component (espressif/mqtt).” [Online]. Available: mqtt documentation

[3] R. Barry, “Mastering the FreeRTOS Real Time Kernel: A Hands-On Tutorial Guide.” Real Time Engineers Ltd.

[4] Wokwi, “Wokwi Simulator Documentation.” [Online]. Available: https://docs.wokwi.com/

[5] FreeRTOS, “FreeRTOS Documentation.” [Online]. Available: freertos documentation

[6] S. Ramírez, “FastAPI Documentation.” [Online]. Available: https://fastapi.tiangolo.com/

[7] SQLite Consortium, “SQLite Documentation.” [Online]. Available: https://www.sqlite.org/docs.html

[8] Eclipse Foundation, “Eclipse Paho MQTT Python Client.” [Online]. Available: https://eclipse.dev/paho/

[9] Object Management Group, “OMG Unified Modeling Language (UML), Version 2.5.1,” 2017.